

REST API Security

Main Treats and Best Practices

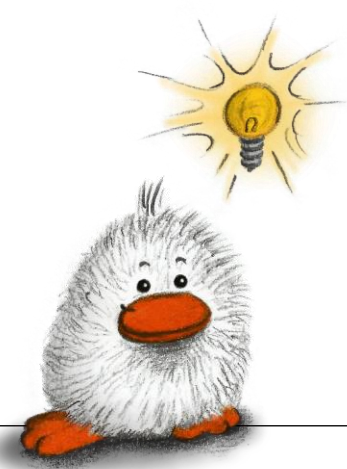
Christian Mainka (@CheariX)

Vladislav Mladenov (@v_mladenov)

Ruhr University Bochum

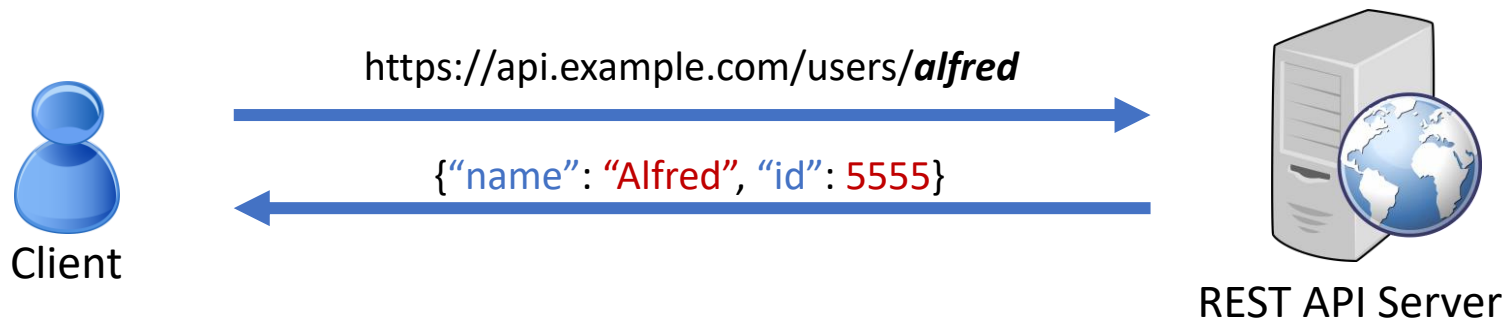


REST API Security

Foundations	REST API
• What is REST? • Where and for what is it used? • How to read OpenAPI files?	

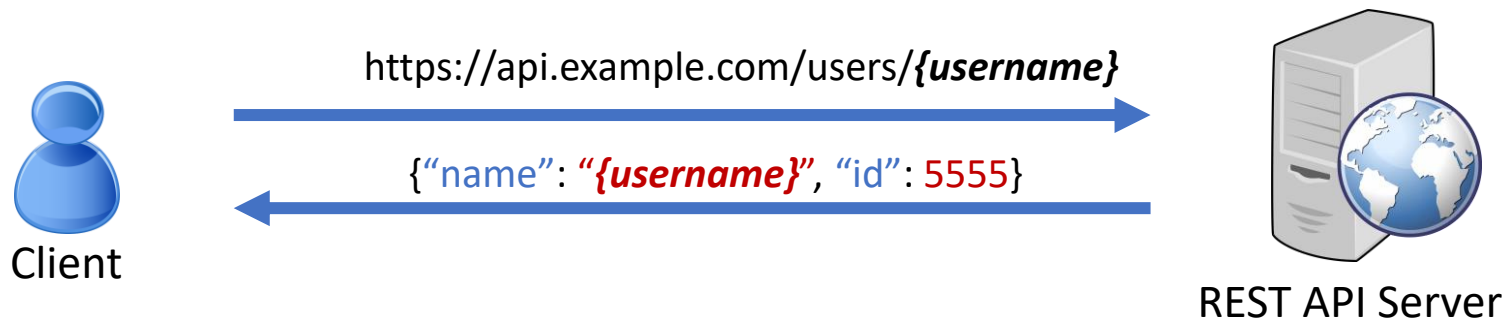
Introduction

- **Representational State Transfer (REST)**
 - Architecture style for web APIs
 - Client: Identify **resources** using parameters
 - Server: Respond with a **representation** of the resource state



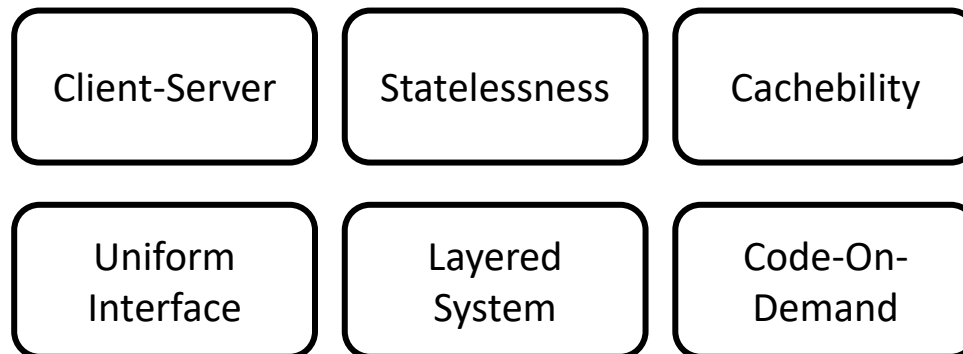
Introduction

- **Representational State Transfer (REST)**
 - Architecture style for web APIs
 - Client: Identify **resources** using parameters
 - Server: Respond with a **representation** of the resource state



What is REST?

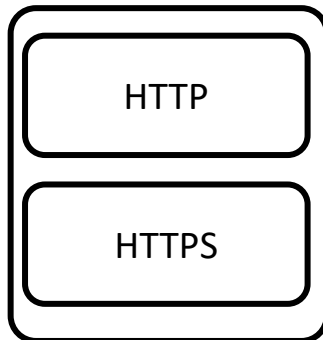
- REST is an architecture style
 - Collection of paradigms
 - No defined protocol
 - Implementations use combinations of common web technologies
 - Goals: Scalability & Efficiency



What is REST?

- RESTful API implementations use established web technologies
 - Depend on combination of web standards
 - Used standards have their own security constraints

Actions & Resource
Identification



HTTP Parameter Pollution

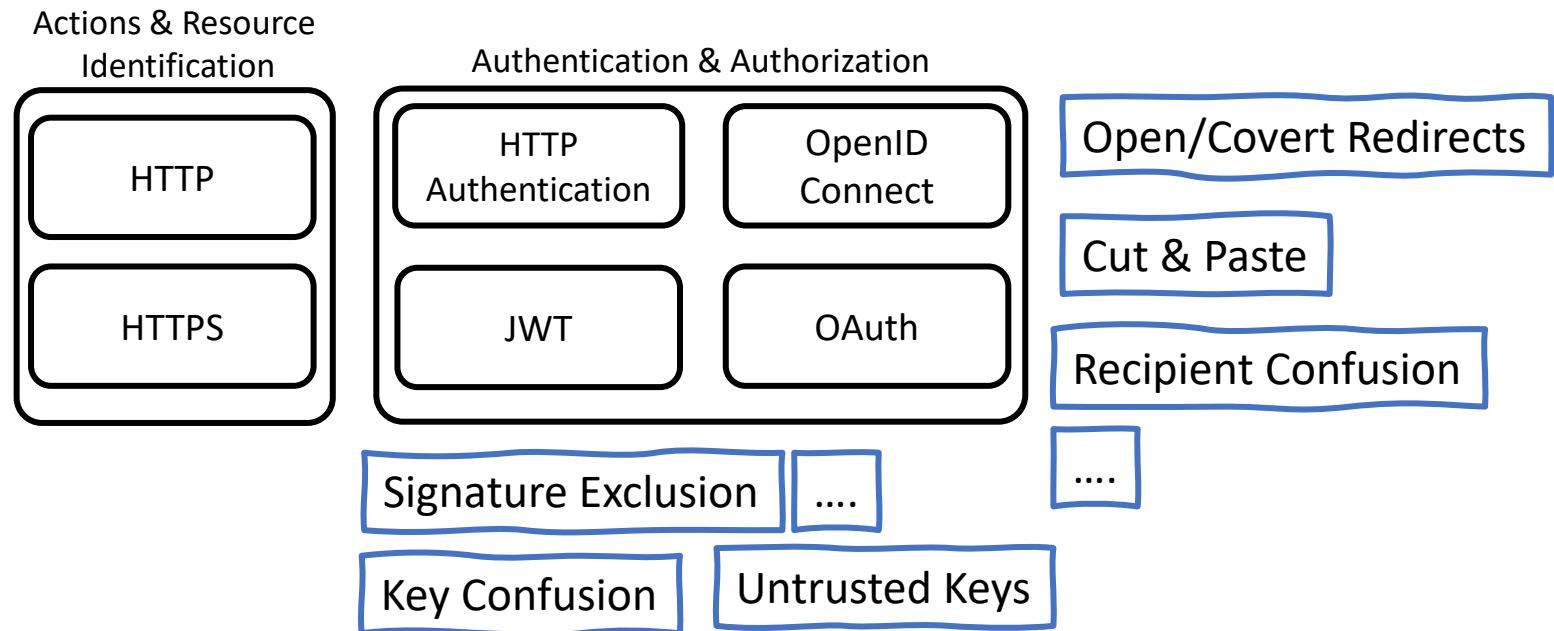
Web Cache Deception Attacks

HTTP Security Headers: HSTS, CSP, CORS,

....

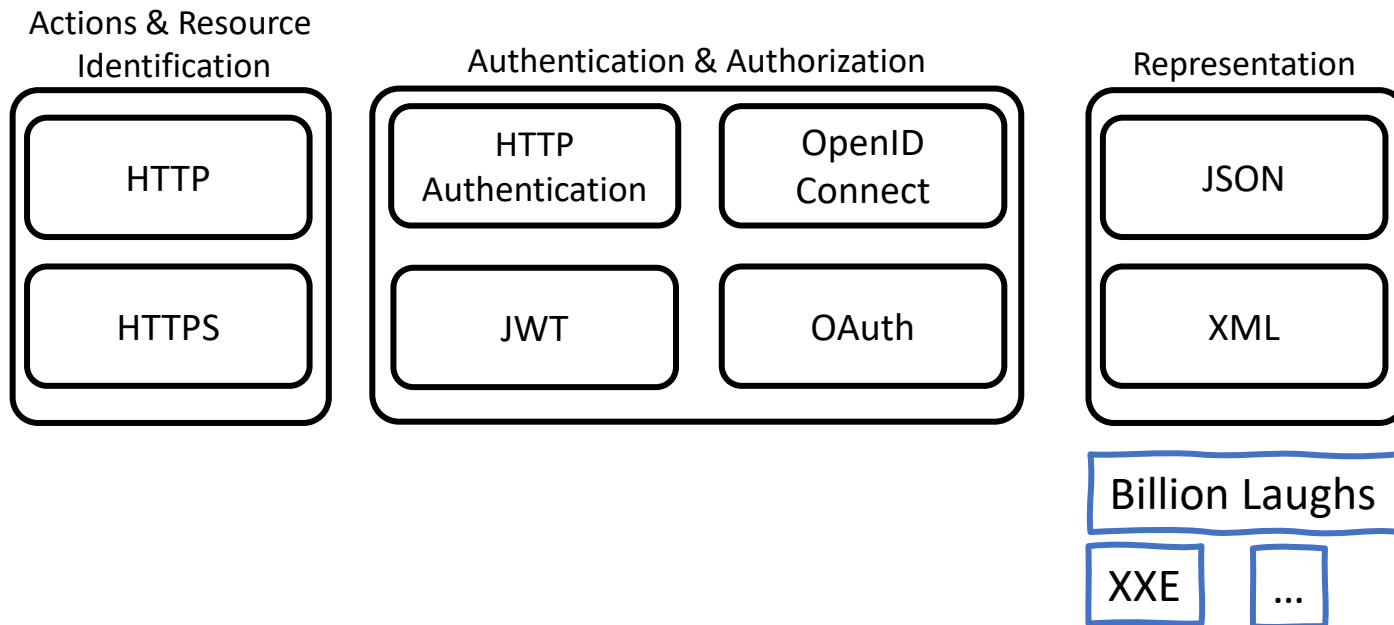
What is REST?

- RESTful API implementations use established web technologies
 - Depend on combination of web standards
 - Used standards have their own security constraints



What is REST?

- RESTful API implementations use established web technologies
 - Depend on combination of web standards
 - Used standards have their own security constraints



Fundamentals

- REST is easy-to-use
 - Uniform interface
 - Interactions can be modelled with HTTP request-response logic

Action

```
GET /users/alfred HTTP/1.1
Host: api.example.com
Authorization: Basic 5aa1bc...129fff
...
```

Endpoint parameters / Resource identifiers

Authentication Info

Client HTTP Request

Fundamentals

- REST is easy-to-use
 - Uniform interface
 - Interactions can be modelled with HTTP request-response logic

Error/Success Code

```
HTTP/1.1 200 OK
Content-Type: application/json
...
{
  "name": "Alfred",
  "id": 5555
}...
```

Representation Metadata

Representation
Data (content)

Server HTTP Response

How to describe a REST API

PayPal Developer

REST APIs

Authentication

Postman Guide

Requests

Responses

CORE RESOURCES

Overview

Add Tracking

Catalog Products

Disputes

Identity

Invoicing

Orders

Google APIs Explorer

Directory Documentation Support

Google APIs Explorer

The Google APIs Explorer is a tool available on most REST API reference documentation pages that lets you try Google API methods without writing code. The APIs Explorer when trying methods that create, modify, or delete data. For more details, read the [APIs Explorer documentation](#).

How to start exploring

1. Click the name of the API you want to explore in the list below. This opens the API reference documentation.
2. In the documentation, on the left, click the method you want to try.
3. On the right, enter the details of your request in the panel labeled "Try this method." Or, click Full screen  for more options.
4. Click **Execute** to send your request to the API and see the API's response.

Google APIs Explorer Directory

Type to filter APIs

Title	Description
Abusive Experience Report API	Views Abusive Experience Report data, and gets a list of sites that have a significant number of abusive experiences.
Accelerated Mobile Pages (AMP) URL API	Retrieves the list of AMP URLs (and equivalent AMP Cache URLs) for a given list of public URL(s).
Access Context Manager API	An API for setting attribute based access control to requests to GCP services.
Ad Exchange Buyer API II	Accesses the latest features for managing Authorized Buyers accounts, Real-Time Bidding configurations and auction metrics, and Marketplace programmatic deals.
Ad Experience Report API	Views Ad Experience Report data, and gets a list of sites that have a significant number of annoying ads.
Admin SDK API 1	Admin SDK lets administrators of enterprise domains to view and manage resources like user, groups etc. It also provides audit and usage reports of domain.
Admin SDK API 1	Admin SDK lets administrators of enterprise domains to view and manage resources like user, groups etc. It also provides audit and usage reports of domain.
Admin SDK API 1	Admin SDK lets administrators of enterprise domains to view and manage resources like user, groups etc. It also provides audit and usage reports of domain.

API

CLI, JavaScript, or cURL. For a

' method and a path.

or body parameters. The API

ly a response body.

, path, and parameters for every

operation. For more information,

"

How to describe a REST API?

PayPal Developer

REST APIs

Authentication

Postman Guide

Requests

Responses

CORE RESOURCES

Overview

Add Tracking

Catalog Products

Disputes

Identity

Invoicing

Orders

Google APIs Explorer

Directory Documentation Support

Google APIs Expl

The Google APIs Explorer is a tool a when trying methods that create, mc

How to start exploring

1. Click the name of the API you
2. In the documentation, on the le
3. On the right, enter the details o
4. Click **Execute** to send your req

Google APIs Explorer Di

Type to filter APIs

Title	Description
Abusive Experience Report API	Views Abus
Accelerated Mobile Pages (AMP) URL API	Retrieves th
Access Context Manager API	An API for s
Ad Exchange Buyer API II	Accesses th
Ad Experience Report API	Views Ad Ex
Admin SDK API 1	Admin SDK
Admin SDK API 1	Admin SDK
Admin SDK API 1	Admin SDK lets administrators of enterprise domains to view and manage resources like user, groups etc. It also provides audit and usage reports of domain.



Machine Readable?

le API methods without writing code. The APIs Explore

more options.

CLI, JavaScript, or cURL. For a

method and a path. or body parameters. The API ly a response body.

, path, and parameters for every operation. For more information,

experiences.

is and auction metrics, and Marketplace programmatic deals.

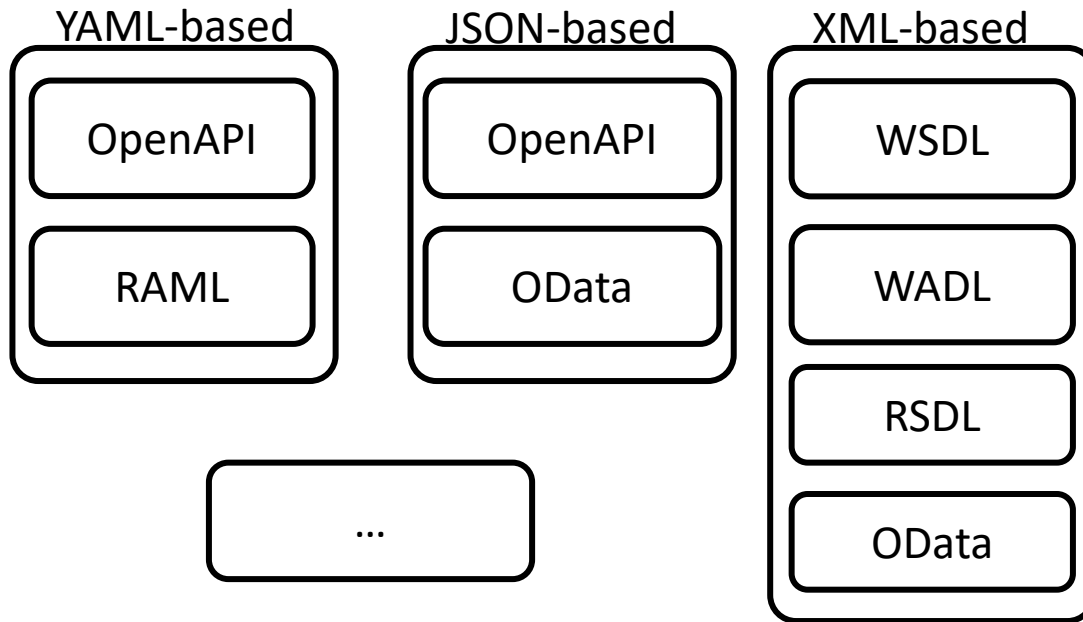
It also provides audit and usage reports of domain.

It also provides audit and usage reports of domain.

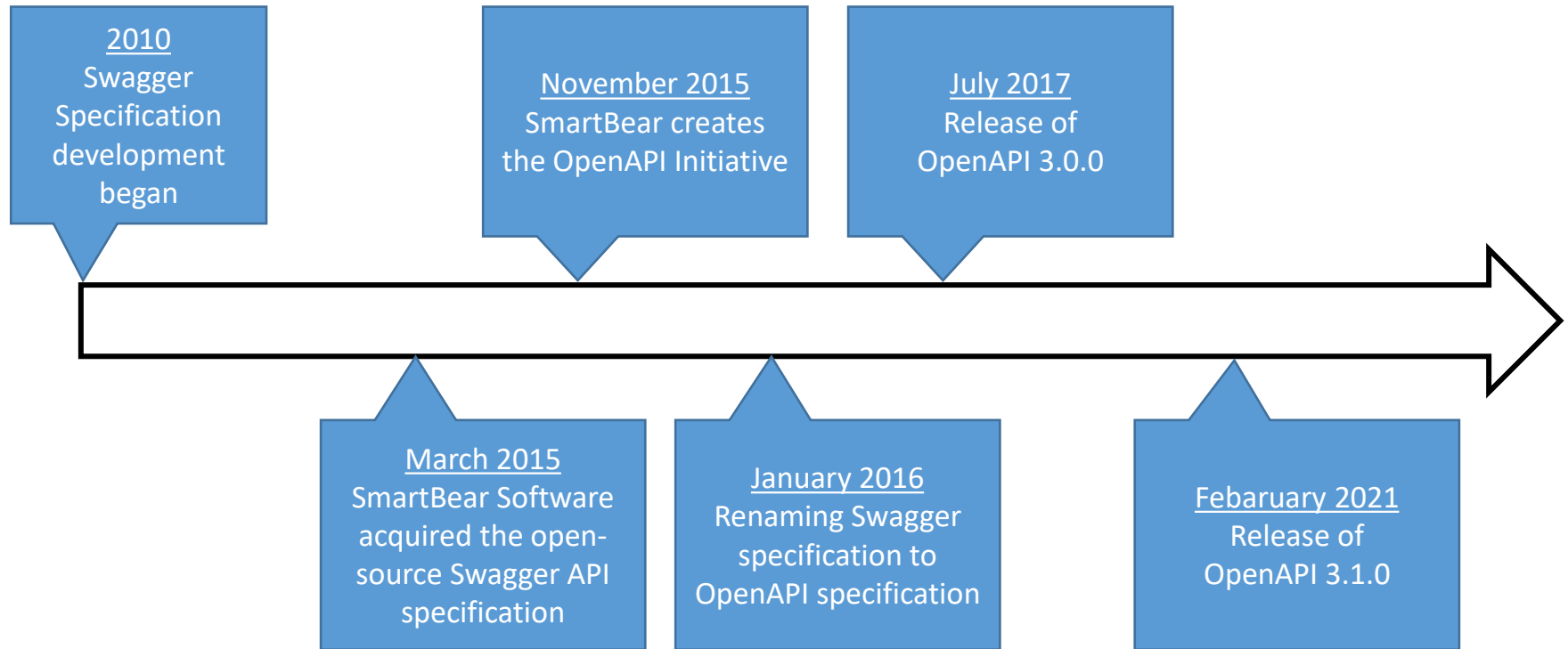
REST API Description Languages

- Goals
 - Formal language providing a structured description
 - Human and machine readable
- Usage
 - Documentation generation
 - Automated generation of software artifacts
 - Code generation
 - Test case generation

REST API Description Languages



OpenAPI: Storyline



OpenAPI: Data Types

- Simple Types

<u>type</u>	<u>format</u>	Comments
integer	int32	signed 32 bits
integer	int64	signed 64 bits (a.k.a long)
number	float	
number	double	
string	password	A hint to UIs to obscure input.

- Rich Text Formatting
 - CommonMark for all **description** fields
- Relative References in URIs
 - Example: "**\$ref**": "**#/components/schemas/userInfo**"

OpenAPI: Zoom Example

Required: Version
number of the OpenAPI
Specification used.

```
openapi: "3.0.0"
  servers: [...]
  info: {...}
  security: [...]
  paths: {...}
  components: {...}
```

OpenAPI: Zoom Example

Required: Version number of the OpenAPI Specification used.

Connectivity information to a target server.



```
openapi: "3.0.0"
servers: [...]
info: {}
security: [...]
paths: {}
components: {}
```

OpenAPI: Zoom Example

Required: Version number of the OpenAPI Specification used.

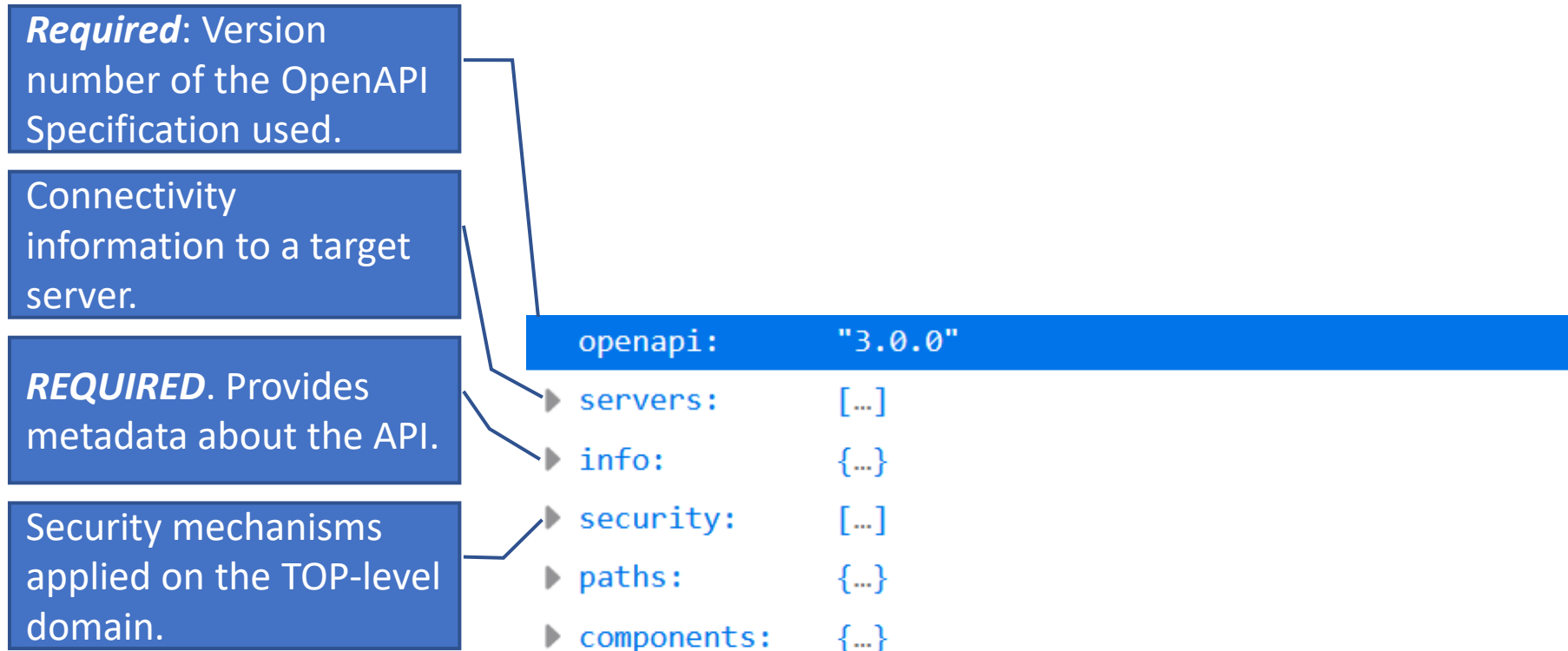
Connectivity information to a target server.

REQUIRED. Provides metadata about the API.

The diagram illustrates the structure of an OpenAPI specification. On the left, three blue boxes provide descriptions for specific fields: 'Required: Version number of the OpenAPI Specification used.' points to 'openapi: "3.0.0"', 'Connectivity information to a target server.' points to 'servers: [...]', and 'REQUIRED. Provides metadata about the API.' points to 'info: {...}'. To the right, a list of fields is shown, each preceded by a right-pointing triangle. The first field, 'openapi: "3.0.0"', is highlighted with a solid blue background, while the others have a light blue background.

openapi:	"3.0.0"
servers:	[...]
info:	{...}
security:	[...]
paths:	{...}
components:	{...}

OpenAPI: Zoom Example



OpenAPI: Zoom Example

Required: Version number of the OpenAPI Specification used.

Connectivity information to a target server.

REQUIRED. Provides metadata about the API.

Security mechanisms applied on the TOP-level domain.

The available paths and operations for the API.

openapi: "3.0.0"

► servers: [...]

► info: {...}

► security: [...]

► paths: {...}

► components: {...}

OpenAPI: Zoom Example

Required: Version number of the OpenAPI Specification used.

Connectivity information to a target server.

REQUIRED. Provides metadata about the API.

Security mechanisms applied on the TOP-level domain.

The available paths and operations for the API.

Holds various reusable schemas for the document.

openapi: "3.0.0"

► servers: [...]

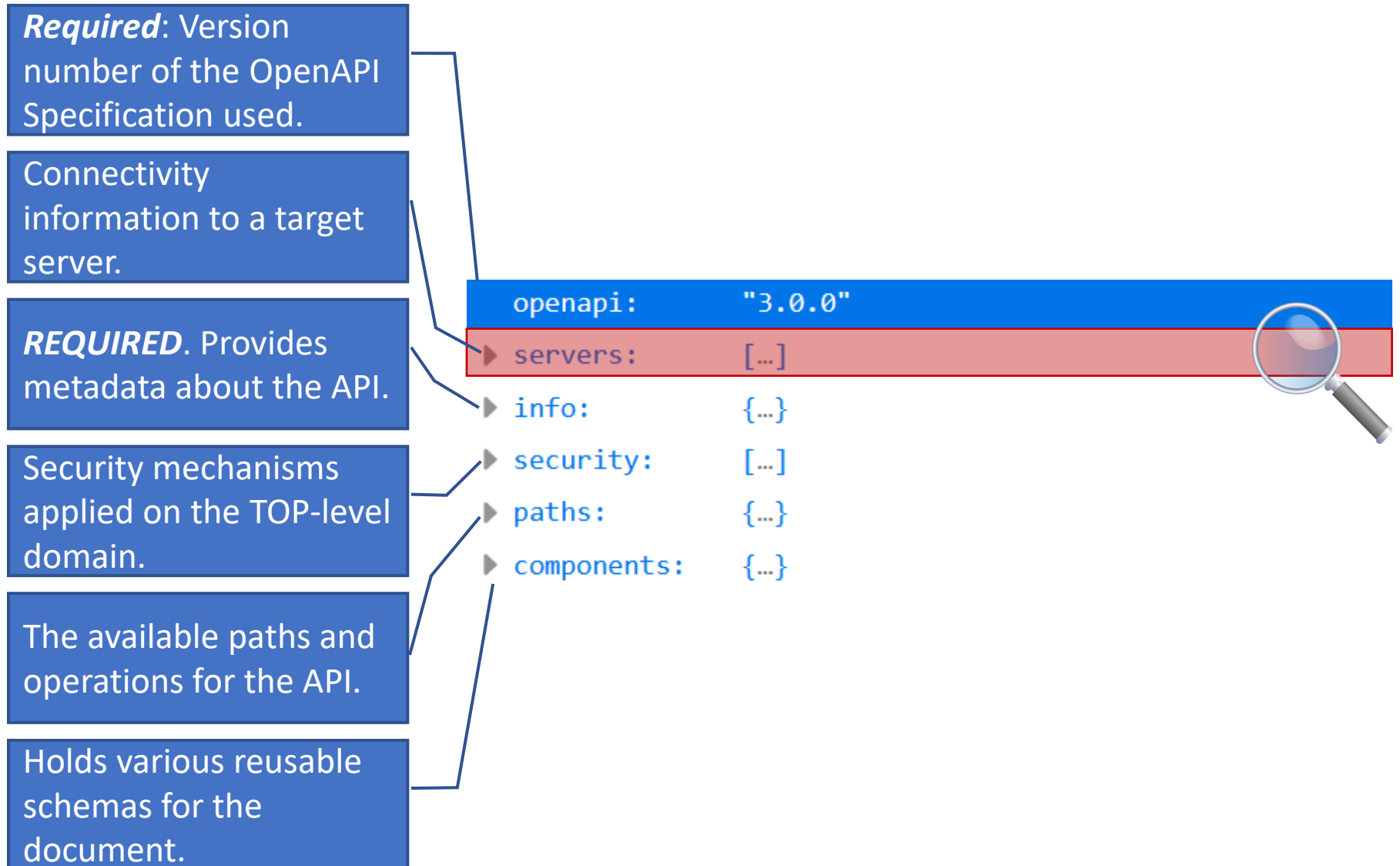
► info: {...}

► security: [...]

► paths: {...}

► components: {...}

OpenAPI: Zoom Example



OpenAPI: Server Object

```
openapi: "3.0.0"
▼ servers:
  ▼ 0:
    url: "https://api.zoom.us/v2"
    ► info: {...}
    ► security: [...]
    ► paths: {...}
    ► components: {...}
```

Absoulte URL of the
server

OpenAPI: Server Object

Absolute URL of the server

Defined variables: *port* and *basePath*

```
{
  "servers": [
    {
      "url": "https://api.server.com:{port}/{basePath}",
      "variables": {
        "port": {
          "enum": ["8443", "443"],
          "default": "8443"
        },
        "basePath": {"default": "v2"}
      }
    }
  ]
}
```

Server Object: Variables Example

OpenAPI: Zoom Example

Required: Version number of the OpenAPI Specification used.

Connectivity information to a target server.

REQUIRED. Provides metadata about the API.

Security mechanisms applied on the TOP-level domain.

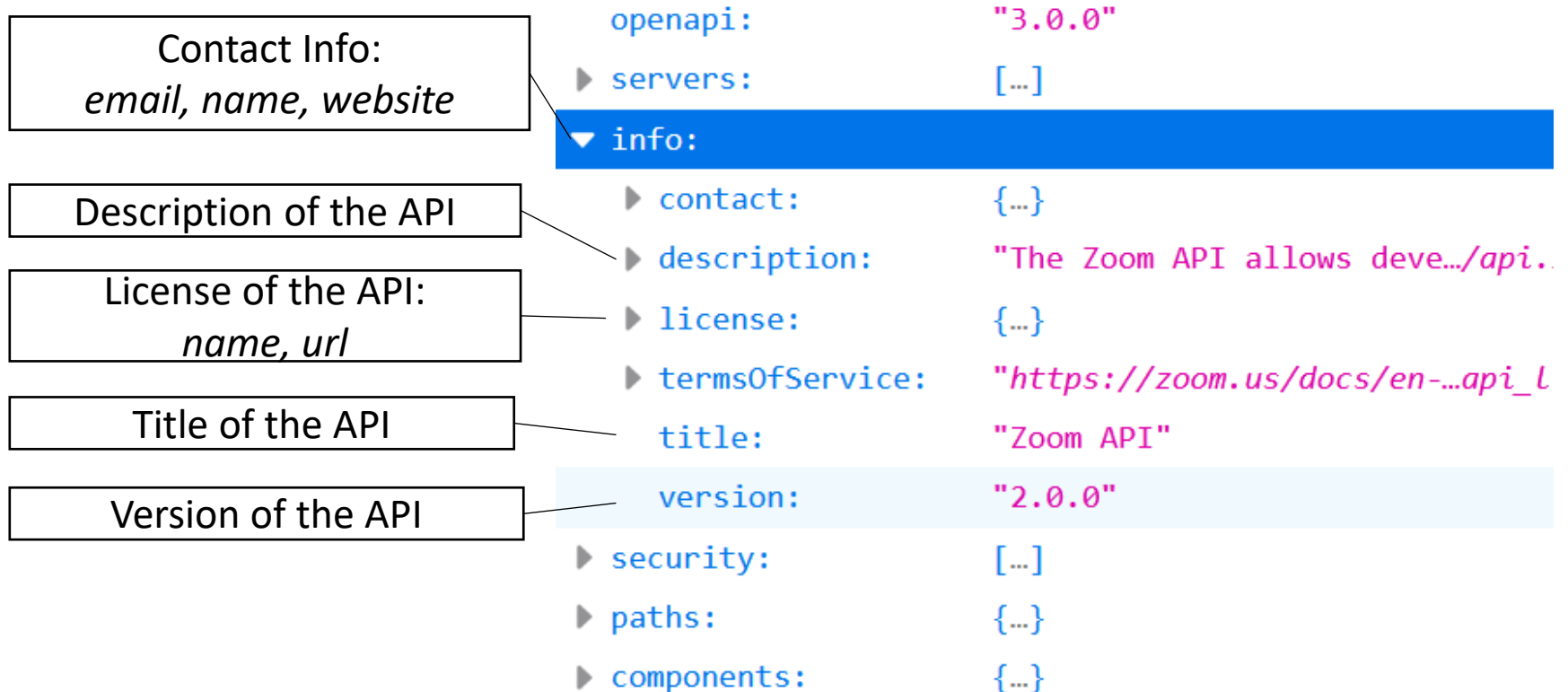
The available paths and operations for the API.

Holds various reusable schemas for the document.

```
openapi: "3.0.0"
servers: [...]
info: {...}
security: [...]
paths: {...}
components: {...}
```



OpenAPI: Info Object



OpenAPI: Zoom Example

Required: Version number of the OpenAPI Specification used.

Connectivity information to a target server.

REQUIRED. Provides metadata about the API.

Security mechanisms applied on the TOP-level domain.

The available paths and operations for the API.

Holds various reusable schemas for the document.

```
openapi: "3.0.0"
servers: [...]
info: {}
security: [...]
paths: {}
components: {}
```



OpenAPI: Security Requirement Object

Top-level declaration of which security mechanisms can be used across the API.

```
openapi: "3.0.0"
servers: [...]
info: {...}
security:
```

▼ 0:

Each name MUST be declared in the *Security Schemes*.

```
  OAuth: []
  paths: {...}
  components: {...}
```

Scope:
e.g. ["read:pets"]
or ["read:pets", "write:pets"]

Individual operations can override this definition.

OpenAPI: Zoom Example

Required: Version number of the OpenAPI Specification used.

Connectivity information to a target server.

REQUIRED. Provides metadata about the API.

Security mechanisms applied on the TOP-level domain.

The available paths and operations for the API.

Holds various reusable schemas for the document.

```
openapi: "3.0.0"
servers: [...]
info: {}
security: [...]
paths: {}
components: {}
```



OpenAPI: Paths Object

```
openapi: "3.0.0"
servers: [...]
info: {...}
security: [...]
▼ paths:
  ▶ /accounts: {...}
  ▶ /accounts/{accountId}: {...}
  ▶ /accounts/{accountId}/billing: {...}
  ▶ /accounts/{accountId}/lock_settings: {...}
  ▶ /accounts/{accountId}/options: {...}
  ▶ /accounts/{accountId}/owner: {...}
  ▶ components: {...}
```

A relative path to an individual endpoint.

OpenAPI: Path Item Object

openapi:	"3.0.0"
▶ servers:	[...]
▶ info:	{...}
▶ security:	[...]
▼ paths:	
▶ /accounts:	{...}
▶ /accounts/{accountId}:	{...}
▶ /accounts/{accountId}/billing:	{...}
▼ /accounts/{accountId}/lock_settings:	
▶ get:	{...}
▶ parameters:	[...]
▶ patch:	{...}
▶ /accounts/{accountId}/options:	{...}
▶ /accounts/{accountId}/owner:	{...}
▶ components:	{...}

Parameters applicable on
all operations.

Operations on this path:
get, post, put, delete,
patch, ...

OpenAPI: Path Item Object

openapi:	"3.0.0"
▶ servers:	[...]
▶ info:	{...}
▶ security:	[...]
▼ paths:	
▶ /accounts:	{...}
▶ /accounts/{accountId}:	{...}
▶ /accounts/{accountId}/billing:	{...}
▼ /accounts/{accountId}/lock_settings:	
▶ get:	{...}
▶ parameters:	[...]
▶ patch:	{...}
▶ /accounts/{accountId}/options:	{...}
▶ /accounts/{accountId}/owner:	{...}
▶ components:	{...}

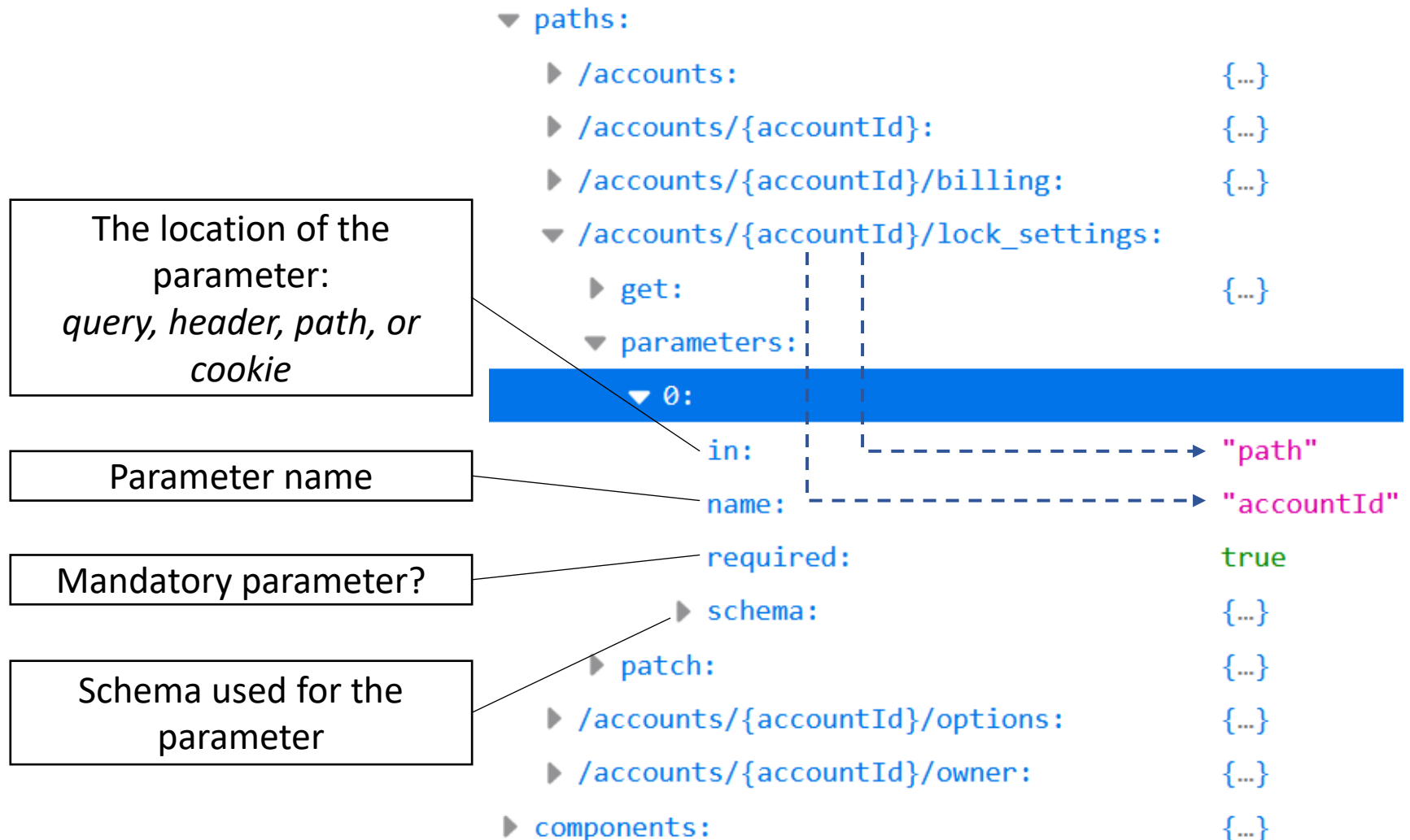
Parameters applicable on all operations.

Operations on this path:
get, post, put, delete, patch, ...

OpenAPI: Parameter Object

openapi:	"3.0.0"
▶ servers:	[...]
▶ info:	{...}
▶ security:	[...]
▼ paths:	
▶ /accounts:	{...}
▶ /accounts/{accountId}:	{...}
▶ /accounts/{accountId}/billing:	{...}
▼ /accounts/{accountId}/lock_settings:	
▶ get:	{...}
▼ parameters:	
▼ 0:	
in:	"path"
name:	"accountId"
required:	true
▶ schema:	{...}
▶ patch:	{...}
▶ /accounts/{accountId}/options:	{...}
▶ /accounts/{accountId}/owner:	{...}

OpenAPI: Parameter Object



OpenAPI: Path Item Object

<code>openapi:</code>	<code>"3.0.0"</code>
▶ <code>servers:</code>	<code>[...]</code>
▶ <code>info:</code>	<code>{...}</code>
▶ <code>security:</code>	<code>[...]</code>
▼ <code>paths:</code>	
▶ <code>/accounts:</code>	<code>{...}</code>
▶ <code>/accounts/{accountId}:</code>	<code>{...}</code>
▶ <code>/accounts/{accountId}/billing:</code>	<code>{...}</code>
▼ <code>/accounts/{accountId}/lock_settings:</code>	
▶ <code>get:</code>	<code>{...}</code>
▶ <code>parameters:</code>	<code>[...]</code>
▶ <code>patch:</code>	<code>{...}</code>
▶ <code>/accounts/{accountId}/options:</code>	<code>{...}</code>
▶ <code>/accounts/{accountId}/owner:</code>	<code>{...}</code>
▶ <code>components:</code>	<code>{...}</code>

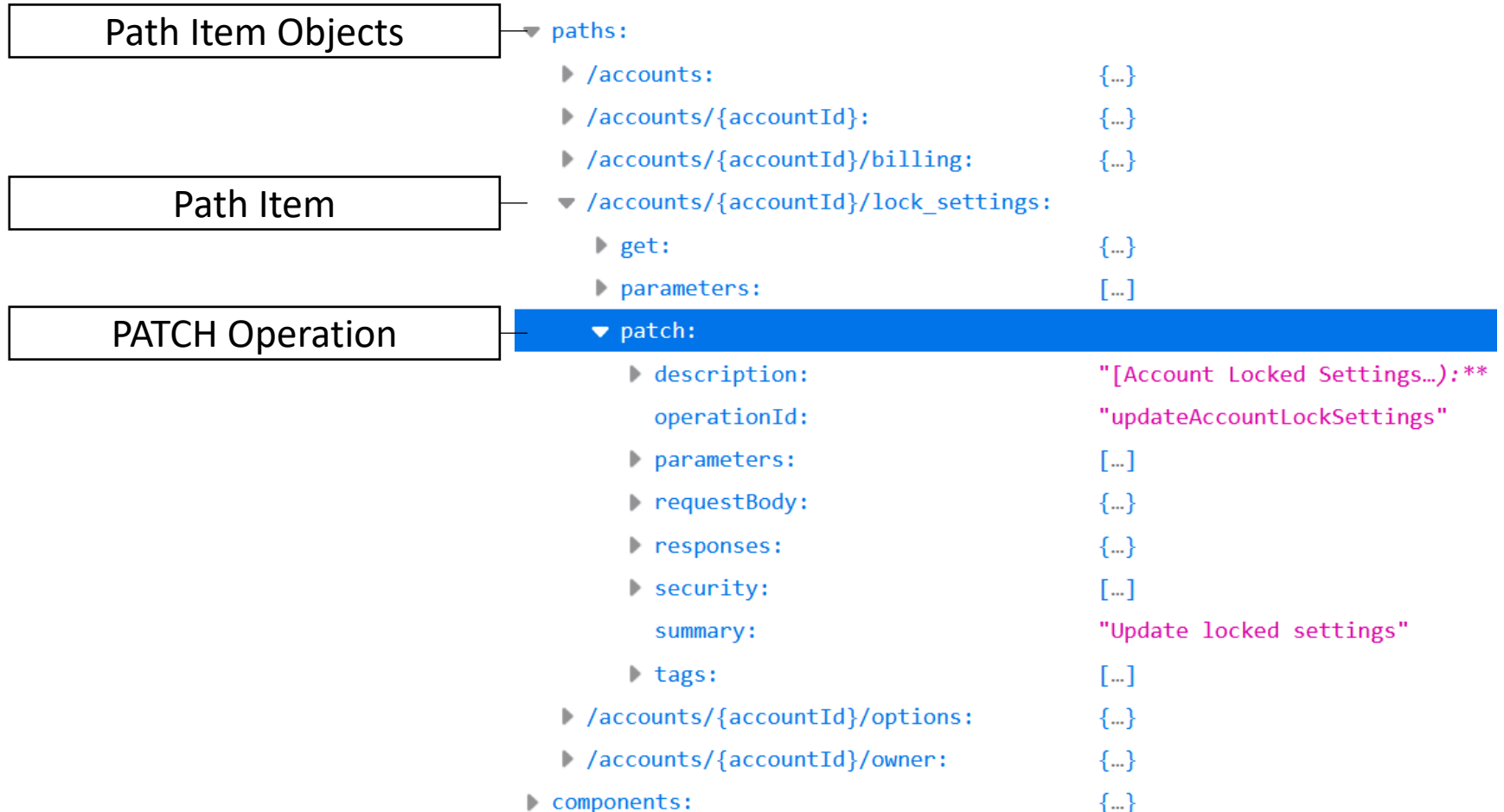
Parameters applicable on all operations.

Operations on this path:
get, post, put, delete, patch, ...

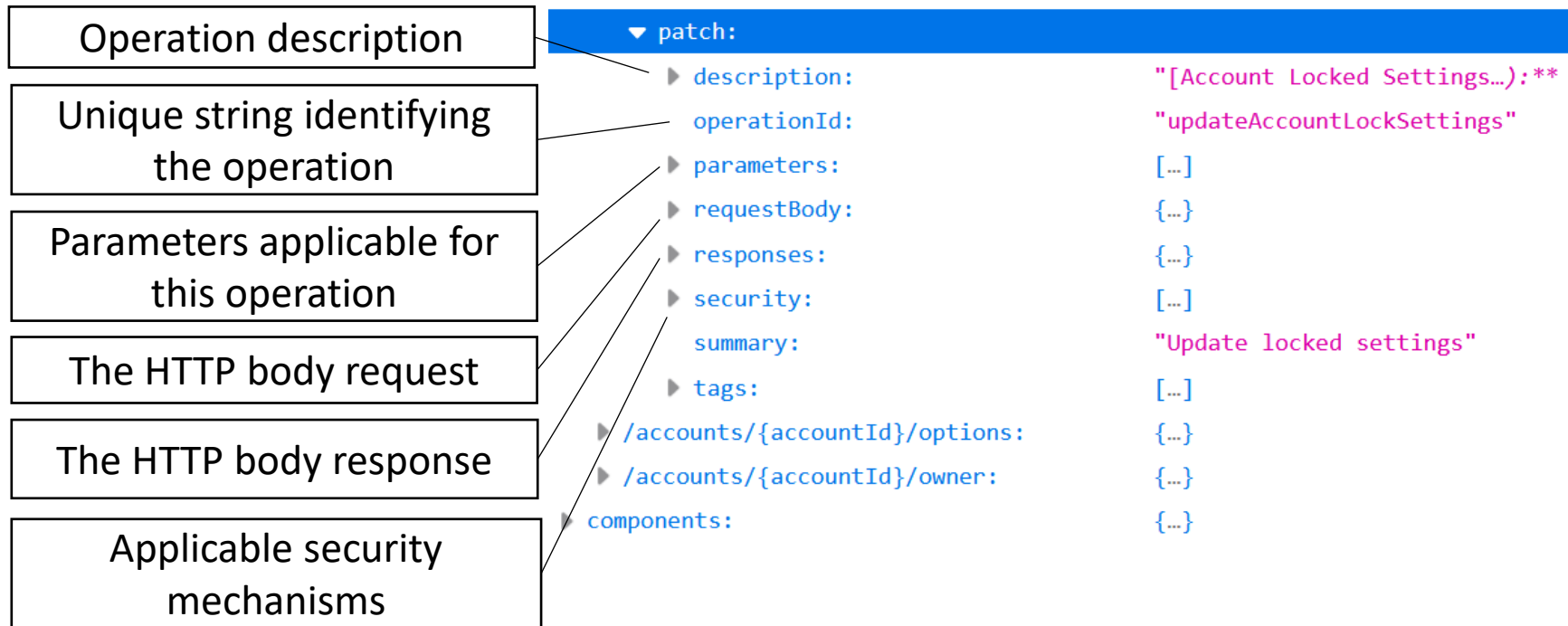
OpenAPI: Operation Object

openapi:	"3.0.0"
▶ servers:	[...]
▶ info:	{...}
▶ security:	[...]
▼ paths:	
▶ /accounts:	{...}
▶ /accounts/{accountId}:	{...}
▶ /accounts/{accountId}/billing:	{...}
▼ /accounts/{accountId}/lock_settings:	
▶ get:	{...}
▶ parameters:	[...]
▼ patch:	
▶ description:	"[Account Locked Settings...]**"
operationId:	"updateAccountLockSettings"
▶ parameters:	[...]
▶ requestBody:	{...}
▶ responses:	{...}
▶ security:	[...]
summary:	"Update locked settings"
▶ tags:	[...]
▶ /accounts/{accountId}/options:	{...}
▶ /accounts/{accountId}/owner:	{...}
▶ components:	{...}

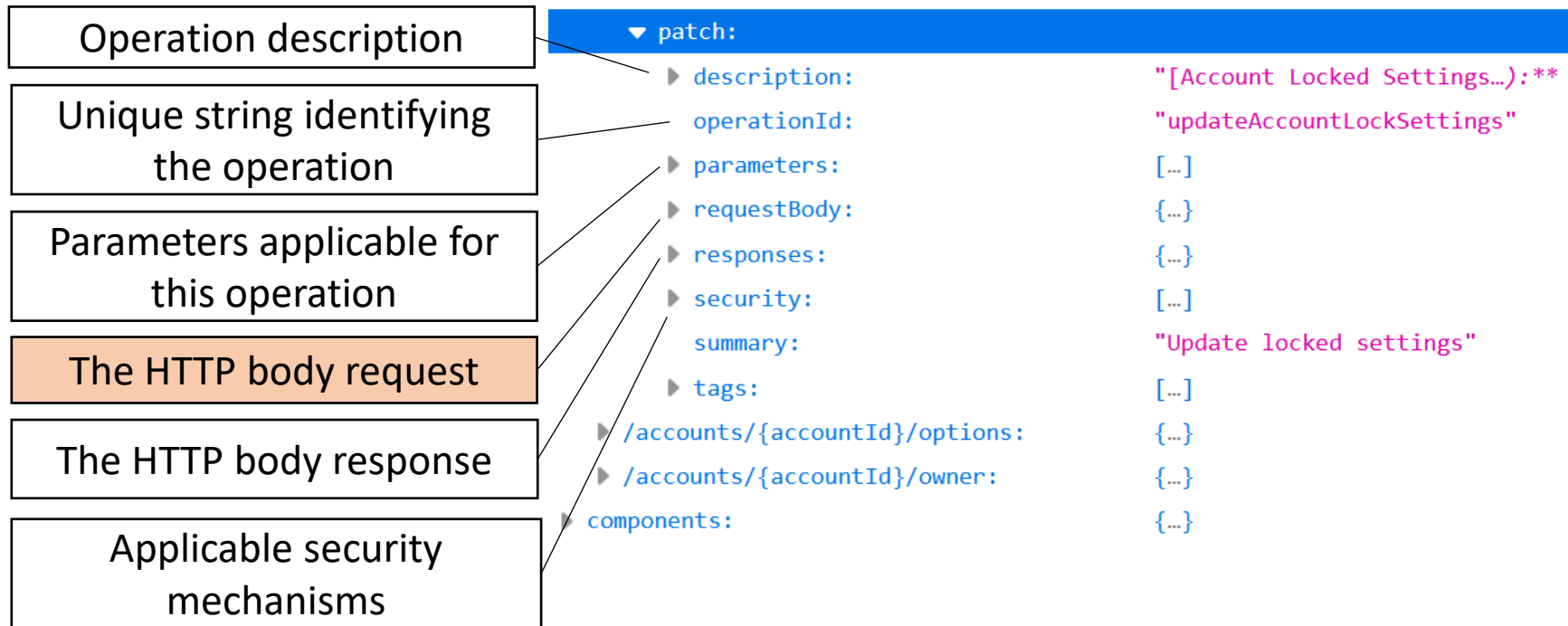
OpenAPI: Operation Object



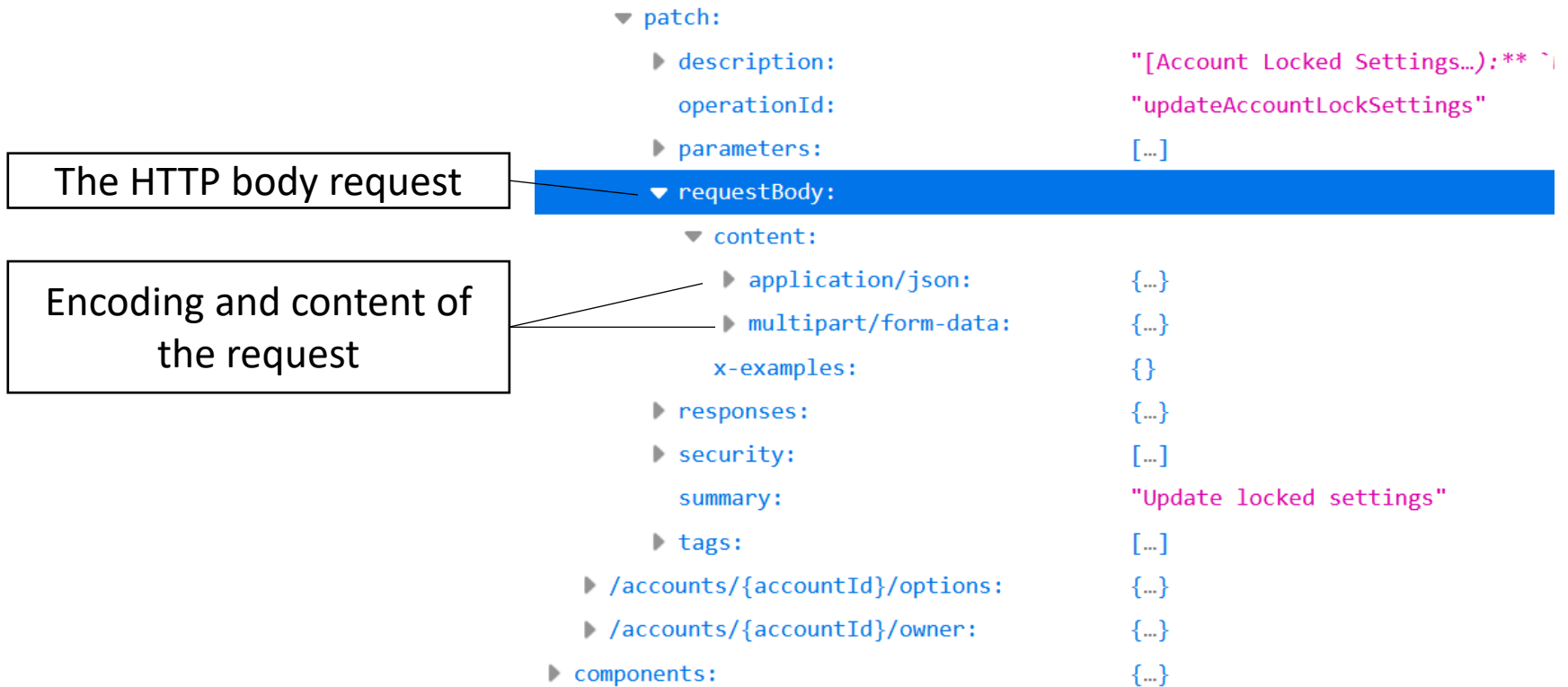
OpenAPI: Operation Object



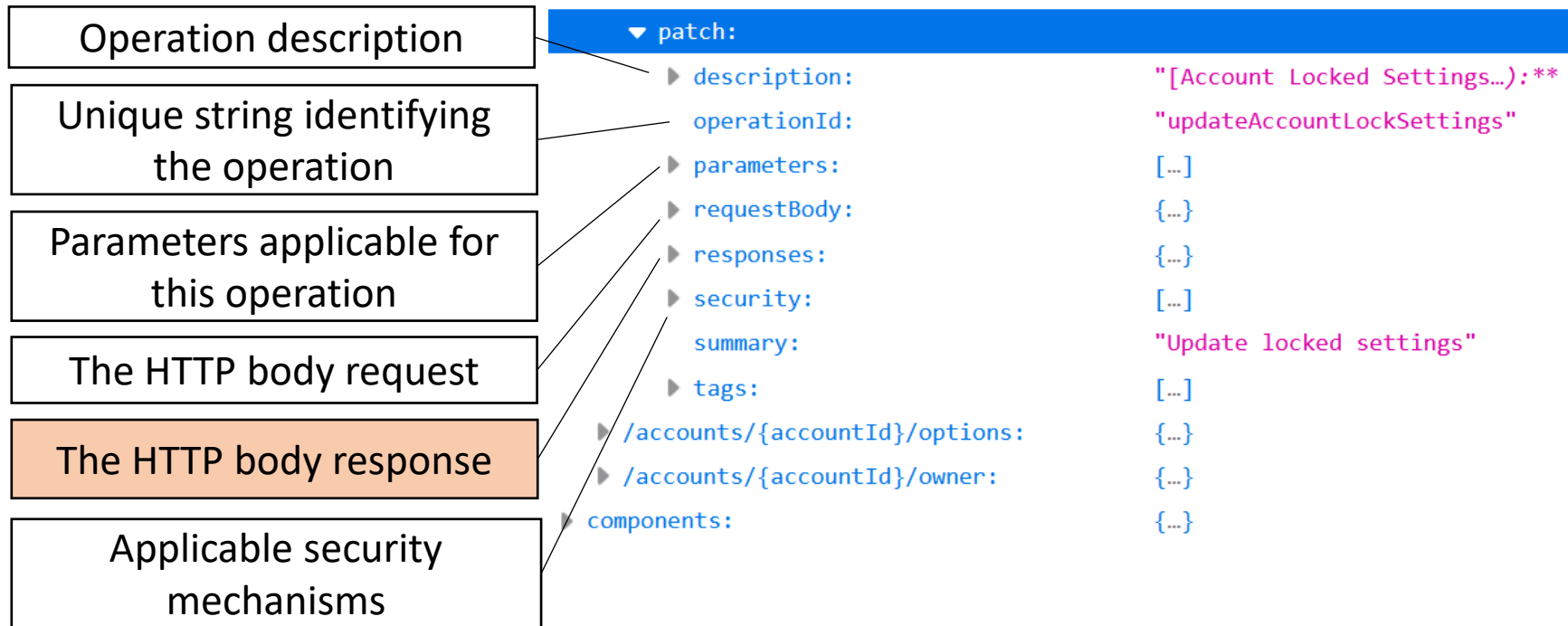
OpenAPI: Operation Object



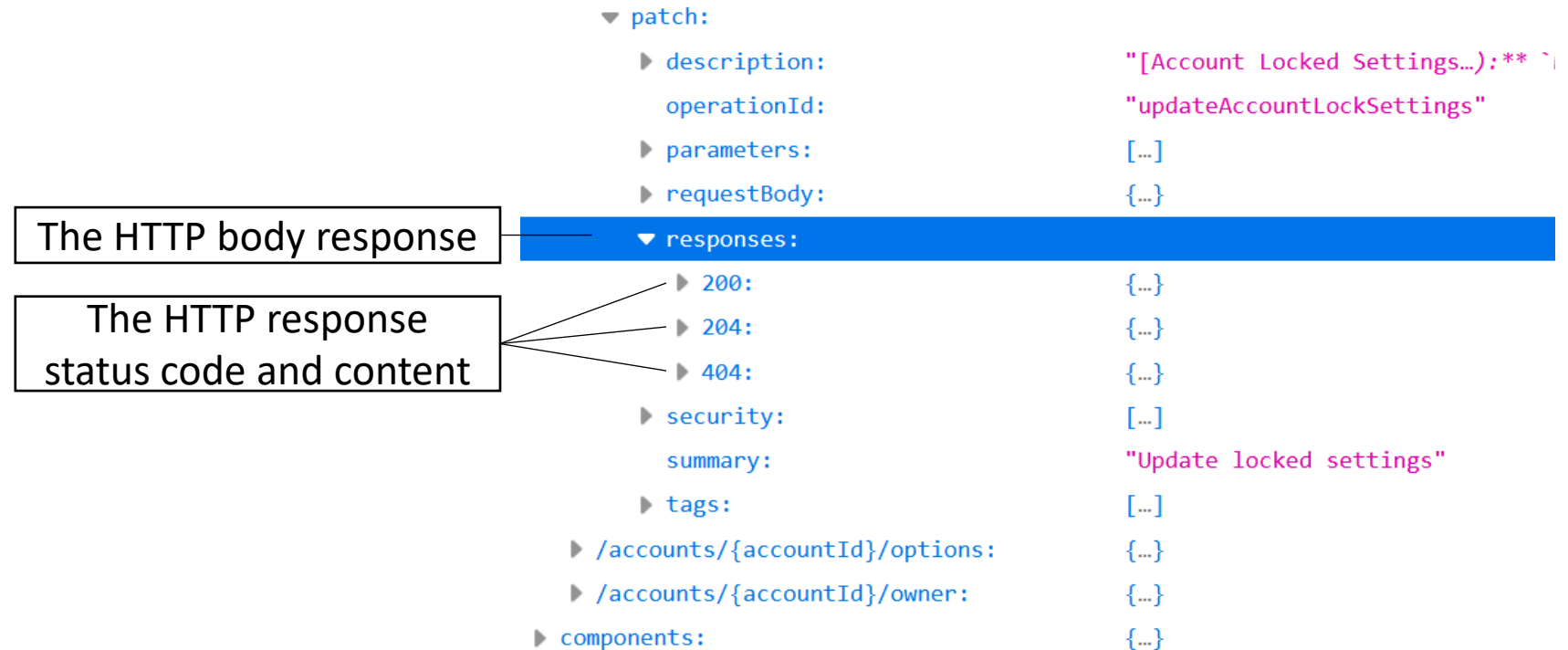
OpenAPI: Operation Object



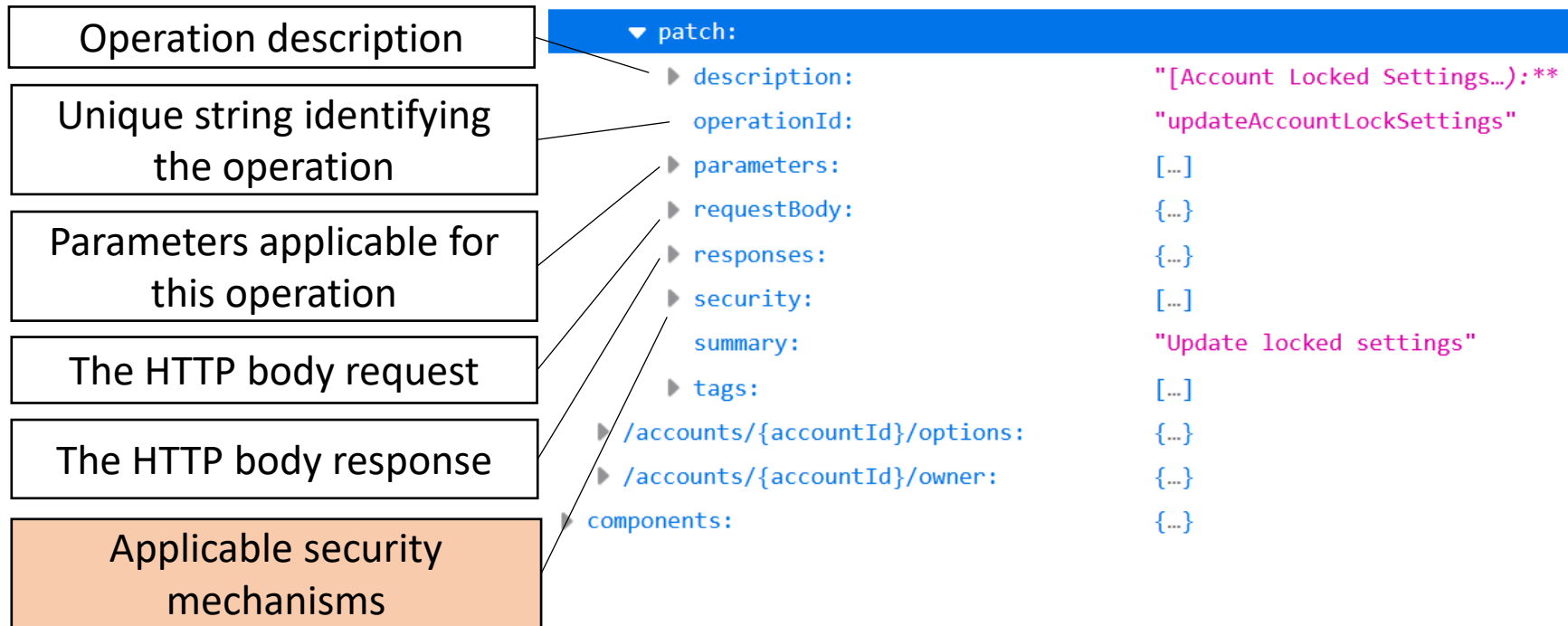
OpenAPI: Operation Object



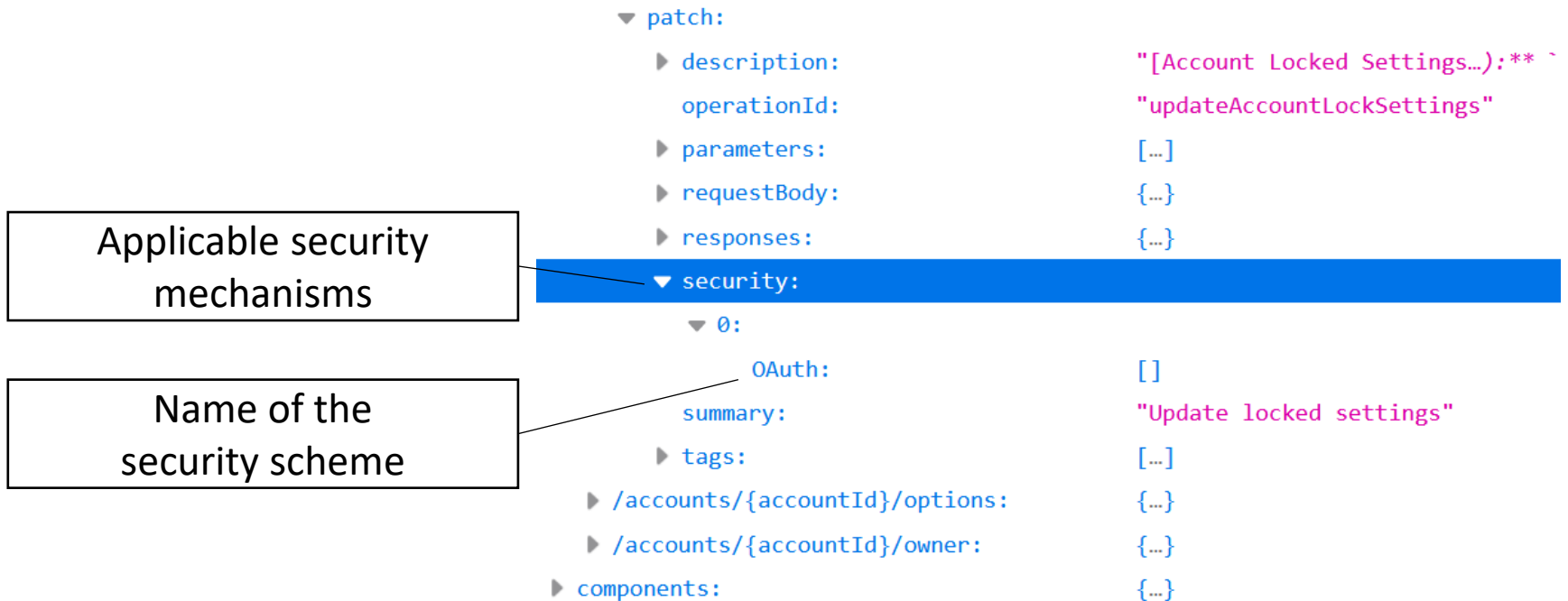
OpenAPI: Operation Object



OpenAPI: Operation Object



OpenAPI: Operation Object



OpenAPI: Zoom Example

Required: Version number of the OpenAPI Specification used.

Connectivity information to a target server.

REQUIRED. Provides metadata about the API.

Security mechanisms applied on the TOP-level domain.

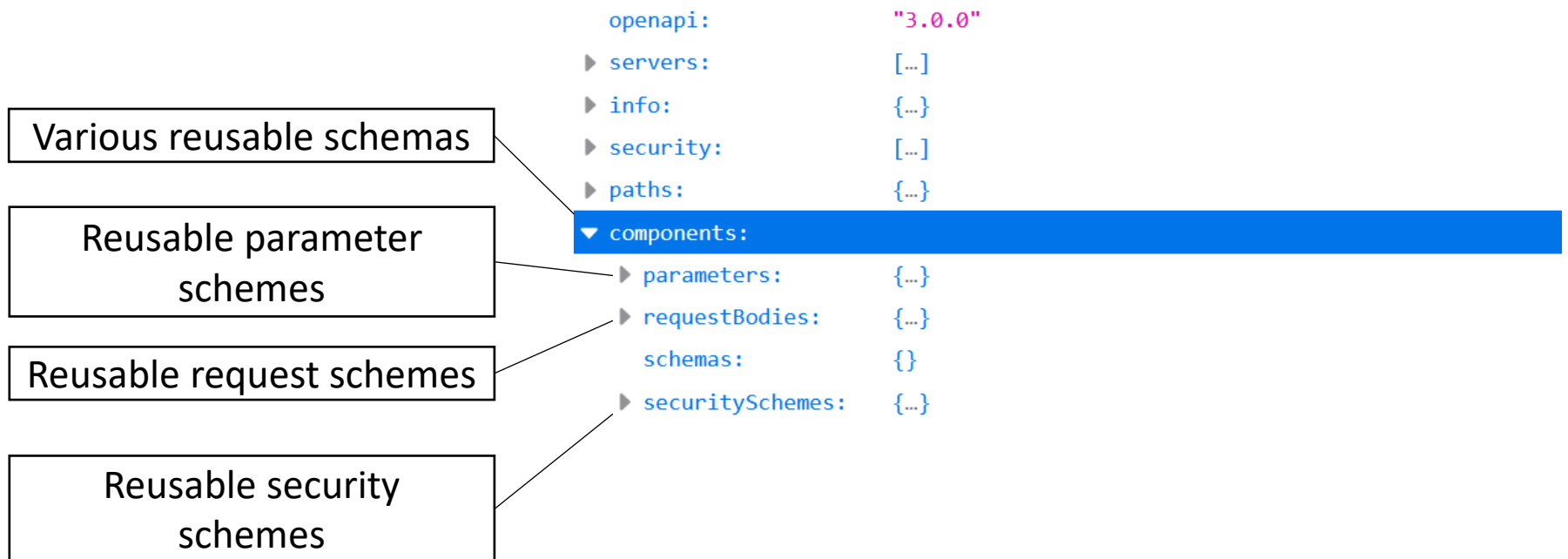
The available paths and operations for the API.

Holds various reusable schemas for the document.

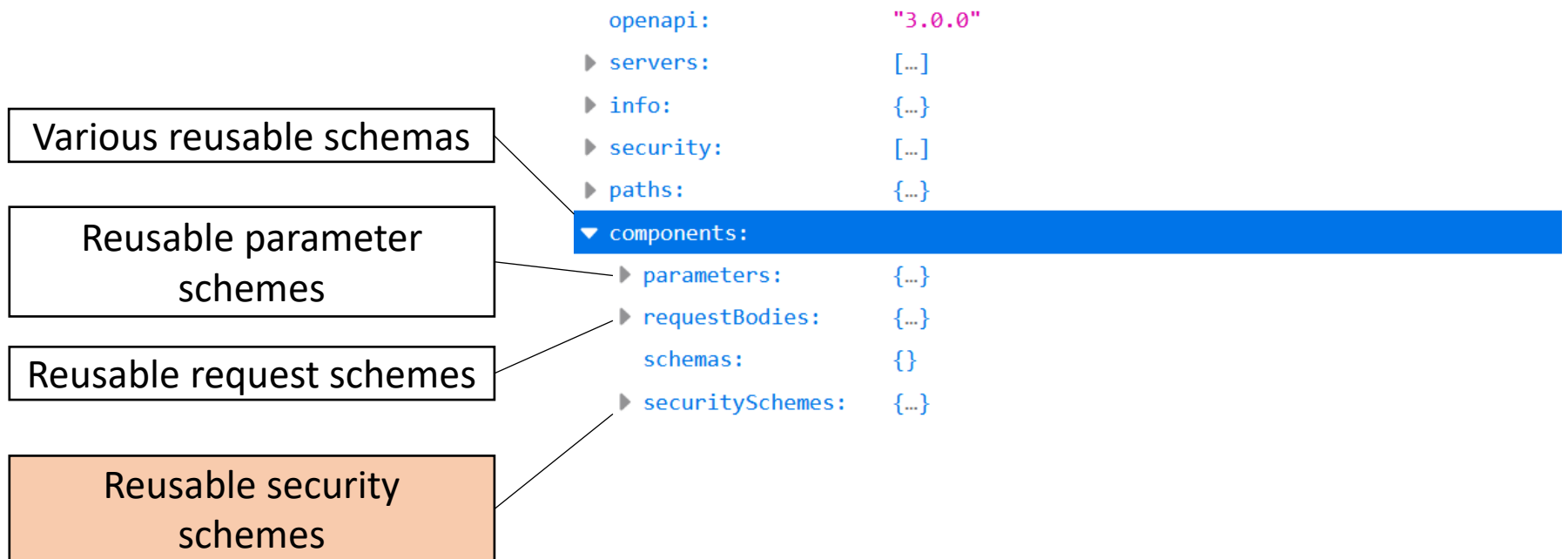
```
openapi: "3.0.0"
servers: [...]
info: {}
security: [...]
paths: {}
components: {}
```



OpenAPI: Components Object



OpenAPI: Components Object



OpenAPI: Security Scheme Object

```
openapi: "3.0.0"
  servers: [...]
  info: {}
  security: [...]
  paths: {}
  components:
    parameters: {}
    requestBodies: {}
    schemas: {}
    securitySchemes:
      Bearer: {}
      Client_Credentials: {}
      OAuth: {}
```

Security scheme *arbitrary*
name: *Bearer*,
Client_Credentials, *OAuth*

OpenAPI: Security Scheme Object

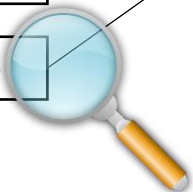
```
openapi: "3.0.0"
servers: [...]
info: {}
security: [...]
paths: {}
components:
  parameters: {}
  requestBodies: {}
  schemas: {}
  securitySchemes:
    Bearer:
      in: "header"
      name: "Authorization"
      type: "apiKey"
      Client_Credentials: {}
      OAuth: {}
```

Name of the scheme

Location of the API key

Name of the *header*,
query or *cookie*
parameter

Security scheme type



REST API Description Languages

Description Language	Type
OpenAPI 3.1.0	http, apiKey (query, header, cookie), oauth2, openIdConnect, mutualTLS

REST API Description Languages

Description Language	Type
OpenAPI 3.1.0	<u>http</u> , apiKey (query, header, cookie), oauth2, openIdConnect, mutualTLS

```
"securitySchemes" :{  
  "example" :{  
    "type": "http",  
    "scheme": "basic"  
  }  
}
```



REST API Description Languages

Description Language	Type
OpenAPI 3.1.0	<u>http</u> , apiKey (query, header, cookie), oauth2, openIdConnect, mutualTLS

```
"securitySchemes" :{  
  "example" :{  
    "type": "http",  
    "scheme": "bearer ",  
    "bearerFormat" : "JWT"  
  }  
}
```



GET /secure
Authorization: Bearer {JWT}



REST API

REST API Description Languages

Description Language	Type
OpenAPI 3.1.0	http, apiKey (query, header, cookie), oauth2, openIdConnect, mutualTLS

```
"securitySchemes" :{  
  "example" :{  
    "type": "apiKey",  
    "name": "api_key",  
    "in": "query"  
  }  
}
```



GET /secure?api_key={value}



REST API

REST API Description Languages

Description Language	Type
OpenAPI 3.1.0	http, apiKey (query, header, cookie), oauth2, openIdConnect, mutualTLS

```
"securitySchemes" :{  
  "example" :{  
    "type": "apiKey",  
    "name": "api_key",  
    "in": "header"  
  }  
}
```



```
GET /secure  
Host: rest-api.com  
api_key: {value}
```



REST API

REST API Description Languages

Description Language	Type
OpenAPI 3.1.0	http, apiKey (query, header, cookie), oauth2, openIdConnect, mutualTLS

```
"securitySchemes" :{  
  "example" :{  
    "type": "apiKey",  
    "name": "api_key",  
    "in": "cookie"  
  }  
}
```



GET /secure
Host: rest-api.com
Cookie: api_key={value}



REST API

REST API Description Languages

Description Language	Type
OpenAPI 3.1.0	http, apiKey (query, header, cookie), <u>oauth2</u> , <u>openIdConnect</u> , mutualTLS

```
"securitySchemes" :{  
  "example" :{  
    "type": "oauth2",  
    "flows": {  
      "implicit": {  
        "authorizationUrl": "https://rest-api.com/oauth",  
        "scopes": {  
          "write:pets": "modify pets",  
          "read:pets": "read your pets"  
        }  
      }  
    }  
  }  
}
```

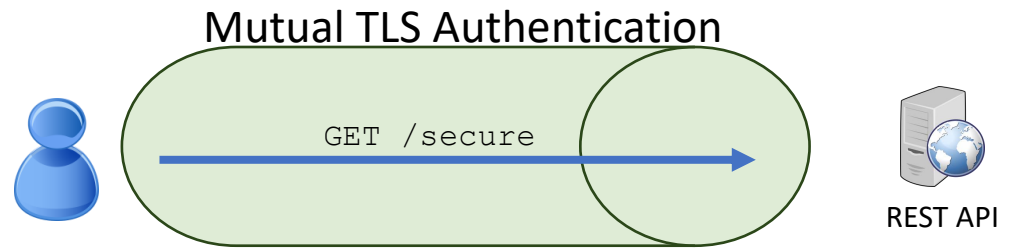
Flow Name	Description
implicit	OAuth Implicit flow
password	OAuth Resource Owner Password flow
Client Credentials	OAuth Client Credentials flow
authorozationCode	OAuth Authorization Code flow

More on this topic later

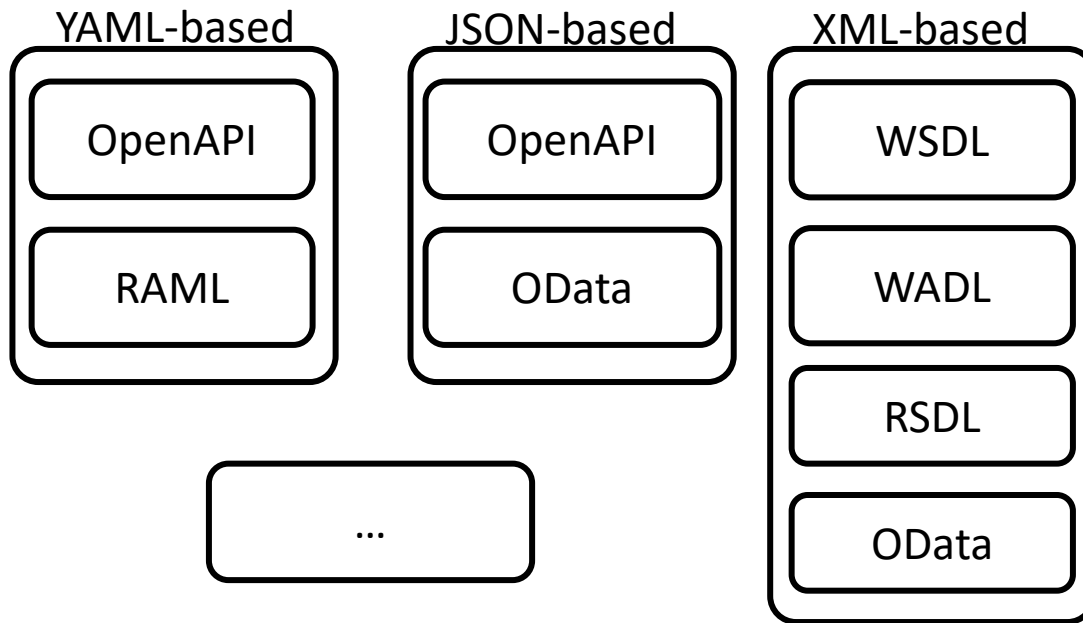
REST API Description Languages

Description Language	Type
OpenAPI 3.1.0	http, apiKey (query, header, cookie), oauth2, openIdConnect, <u>mutualTLS</u>

```
"securitySchemes" :{  
  "example" :{  
    "type": "mutualTLS",  
  }  
}
```



REST API Description Languages



REST API Description Languages

Description Language	Type
OpenAPI 2.0	basic, apiKey (query, header), oauth2

REST API Description Languages

Description Language	Type
OpenAPI 2.0	basic, apiKey (query, header), oauth2
OpenAPI 3.0.x	http, apiKey (query, header, cookie), oauth2, openIdConnect
OpenAPI 3.1.0	basic, apiKey (query, header, cookie), oauth2,

REST API Description Languages

Description Language	Type
OpenAPI 2.0	basic, apiKey (query, header), oauth2
OpenAPI 3.0.x	http, apiKey (query, header, cookie), oauth2, openIdConnect
OpenAPI 3.1.0	http, apiKey (query, header, cookie), oauth2, openIdConnect, mutualTLS
RAML 0.8	Basic Authentication. Digest Authentication. OAuth 1.0 .

REST API Description Languages

Description Language	Type
OpenAPI 2.0	basic, apiKey (query, header), oauth2
OpenAPI 3.0.x	http, apiKey (query, header, cookie), oauth2, openIdConnect
OpenAPI 3.1.0	http, apiKey (query, header, cookie), oauth2, openIdConnect, mutualTLS
RAML 0.8	Basic Authentication, Digest Authentication, OAuth 1.0 , OAuth 2.0, x-other
RAML 1.0	Basic Authentication, Digest Authentication, OAuth 1.0,

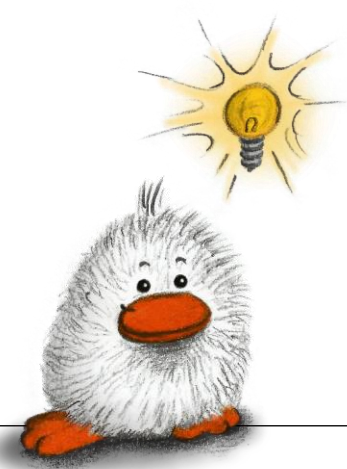
REST API Description Languages

Description Language	Type
OpenAPI 2.0	basic, apiKey (query, header), oauth2
OpenAPI 3.0.x	http, apiKey (query, header, cookie), oauth2, openIdConnect
OpenAPI 3.1.0	http, apiKey (query, header, cookie), oauth2, openIdConnect, mutualTLS
RAML 0.8	Basic Authentication, Digest Authentication, OAuth 1.0 , OAuth 2.0, x-other
RAML 1.0	Basic Authentication, Digest Authentication, OAuth 1.0, OAuth 2.0, x-other, Pass Trough
API Blueprint	Basic, OAuth 2

REST API Description Languages

Description Language	Type
OpenAPI 2.0	basic, apiKey (query, header), oauth2
OpenAPI 3.0.x	http, apiKey (query, header, cookie), oauth2, openIdConnect
OpenAPI 3.1.0	http, apiKey (query, header, cookie), oauth2, openIdConnect, mutualTLS
RAML 0.8	Basic Authentication, Digest Authentication, OAuth 1.0 , OAuth 2.0, x-other
RAML 1.0	Basic Authentication, Digest Authentication, OAuth 1.0, OAuth 2.0, x-other, Pass Trough
API Blueprint	Basic, OAuth 2

REST API Security

Security Overview	REST API
• OWASP API Security Top 10 • Best Current Practices • On-going Research	

OWASP API Security Top 10 (2023)

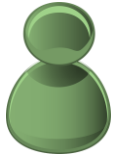
- Broken object level authorization
- Broken authentication
- Broken object property level authorization
- Unrestricted resource consumption
- Broken function level authorization
- Unrestricted Access to Sensitive Business Flows
- Server Side Request Forgery
- Security Misconfiguration
- Improper Inventory Management
- Unsafe Consumption of APIs

Fictional example



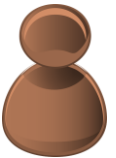
Customer

- Can **read** the info of products
- Can **buy** products



Seller

- Can **manage (create/delete/change)** products in the **own** shop



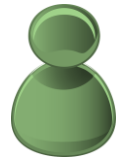
Admin

- Can **manage (create/delete/change)** users

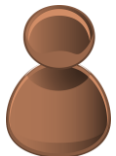


Insecure Shop
REST API Server

Fictional example



Seller



Admin

Servers

<https://shop.mls.org/v2.0/production> ▼

Authorize



Customer Area ^

GET /{product_id} ▼

POST /buy ▼ 

Seller Area ^

GET /{shop_id}/{product_id} ▼ 

PUT /{shop_id}/{product_id} ▼ 

Admin Area ^

GET /admins/{user_id} ▼ 

PATCH /admins/{user_id} ▼

DELETE /admins/{user_id} ▼ 



Insecure Shop
REST API Server

OWASP API Security Top 10



Broken object level authorization

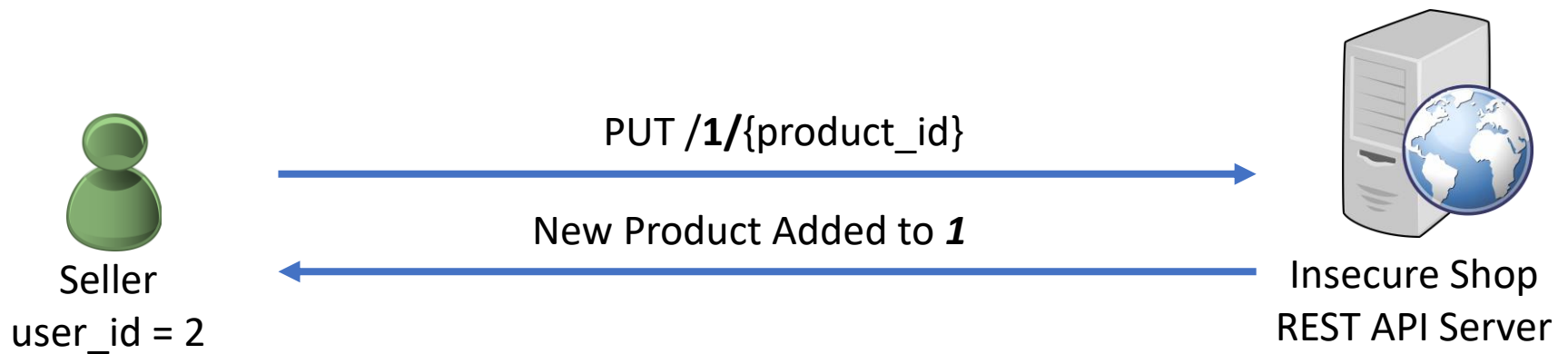
- Broken authentication
- Broken object property level authorization
- Unrestricted resource consumption
- Broken function level authorization
- Unrestricted Access to Sensitive Business Flows
- Server Side Request Forgery
- Security Misconfiguration
- Improper Inventory Management
- Unsafe Consumption of APIs

Broken object level authorization



User	user_id	shop_id	Role
	1	-	customer
	2	1	seller
	3	-	admin

Broken object level authorization



User	user_id	shop_id	Role
	1	-	customer
	2	1	seller
	3	-	admin

Broken object level authorization

Unauthorized access by manipulating the ID of an object.



Seller
user_id = 2

PUT /**4**/product_id

Application Logic
put(shop_id, product_id)

New Product Added to **shop_id = 4**

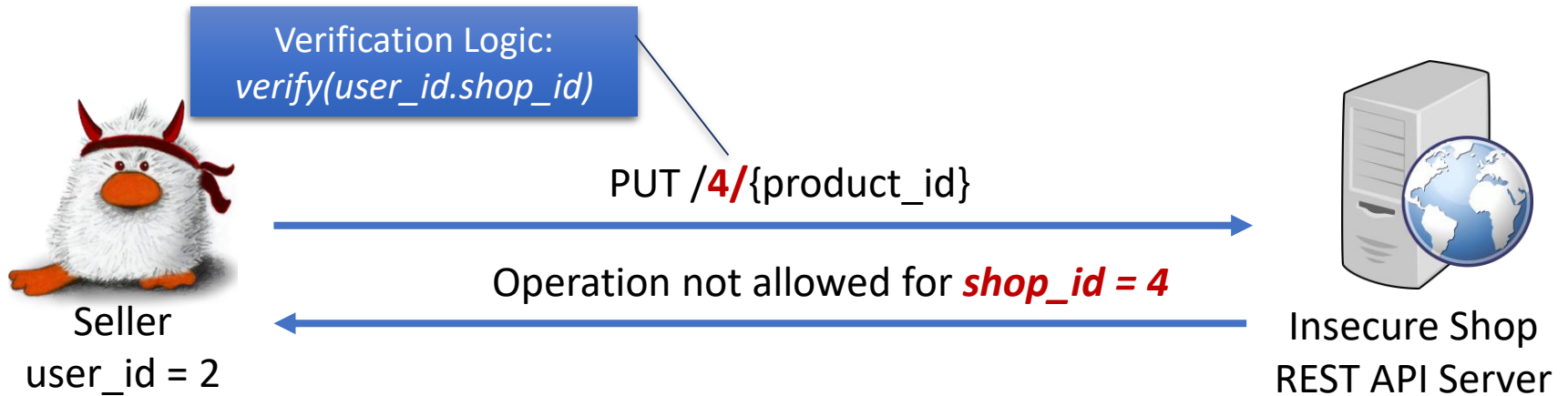


Insecure Shop
REST API Server

User	user_id	shop_id	Role
	1	-	customer
	2	1	seller
	3	-	admin

Broken object level authorization

Unauthorized access by manipulating the ID of an object.

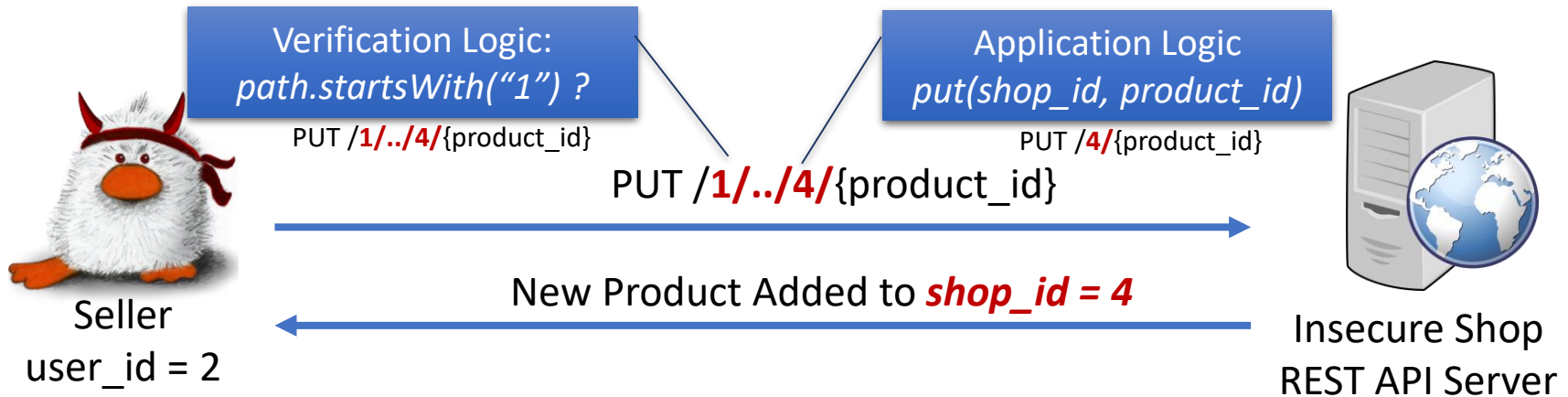


User	user_id	shop_id	Role
	1	-	customer
	2	1	seller
	3	-	admin

Broken object level authorization

PATH TRAVERSAL

Unauthorized access by manipulating the ID of an object.



User	user_id	shop_id	Role
	1	-	customer
	2	1	seller
	3	-	admin

Broken object level authorization

ENUMERATION ATTACK

Unauthorized access by manipulating the ID of an object.



Seller
user_id = 2

PUT /**1...n**/product_id

New Product Added

Forbidden

Forbidden

Error: Shop does not exist

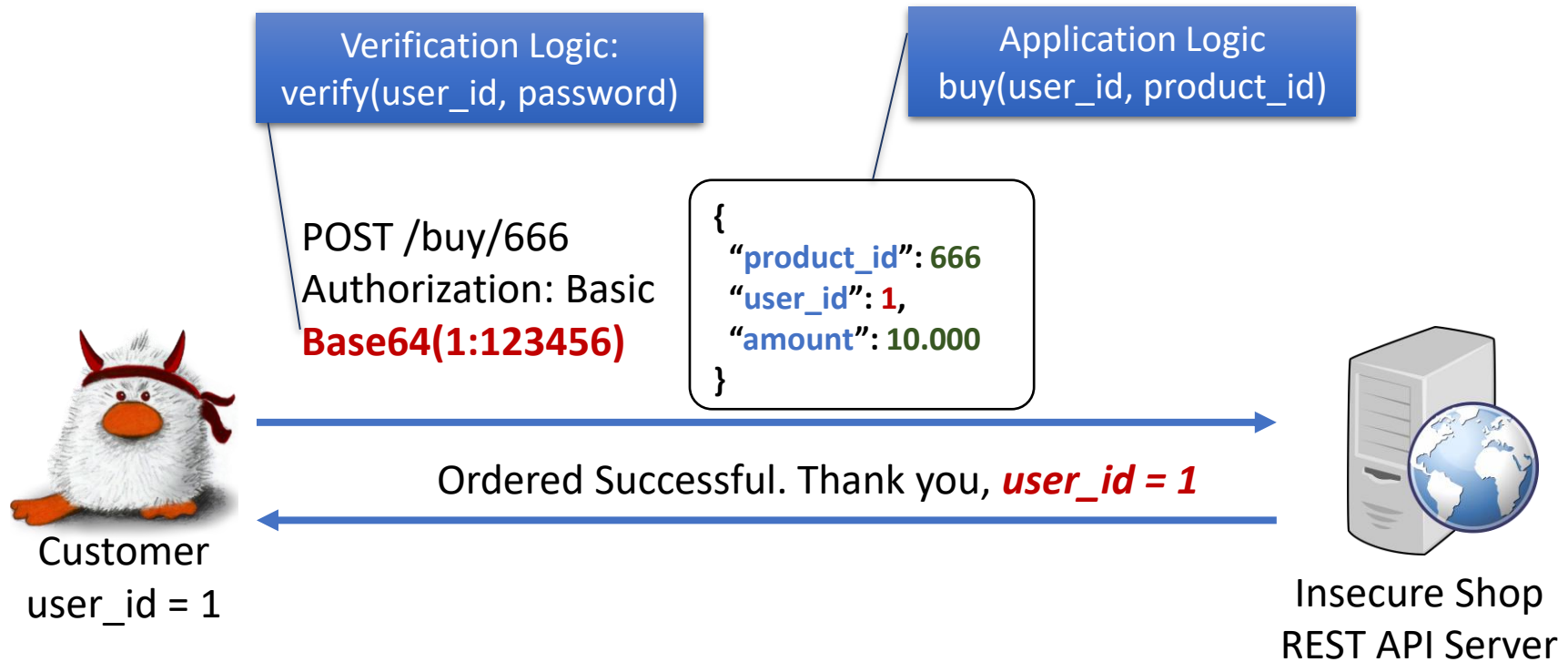


Insecure Shop
REST API Server

Broken object level authorization

REDUNDANT PARAMETERS

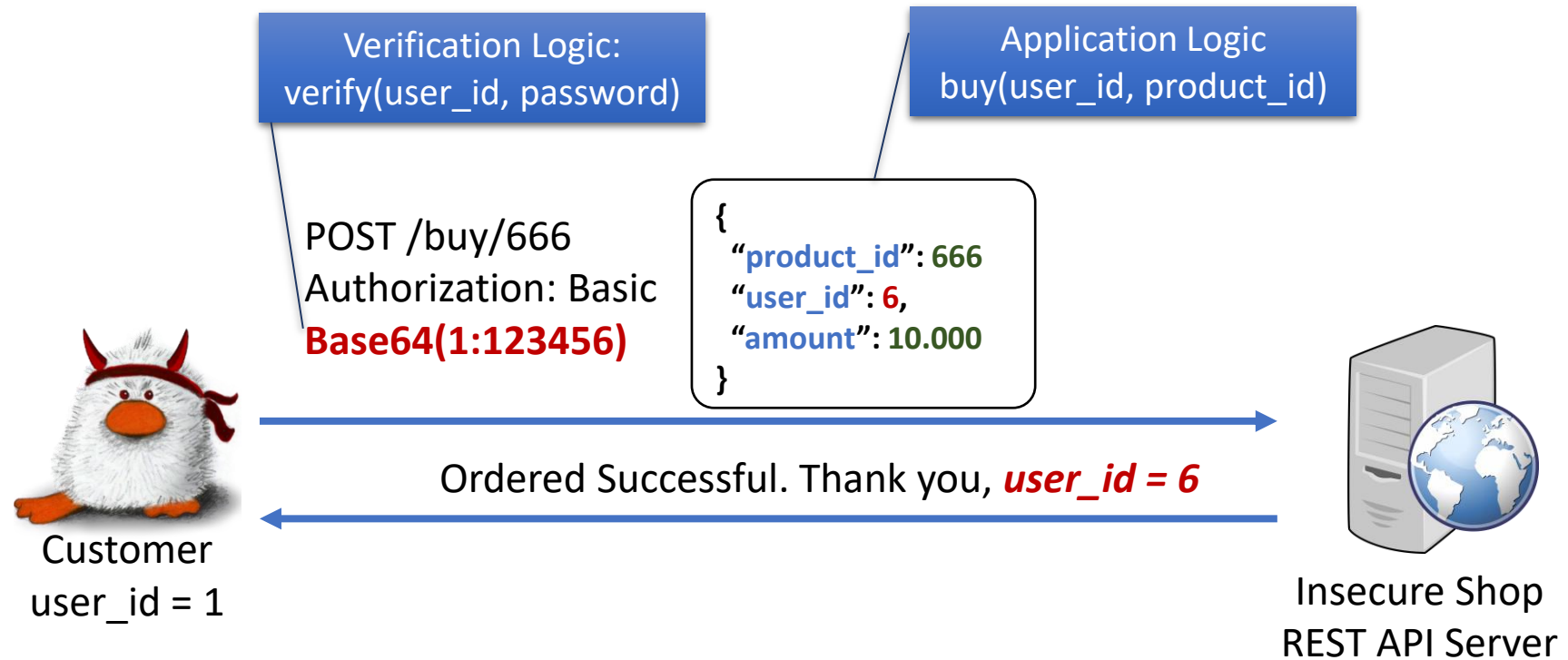
Unauthorized access by manipulating the ID of an object.



Broken object level authorization

REDUNDANT PARAMETERS

Unauthorized access by manipulating the ID of an object.



OWASP API Security Top 10

- Broken object level authorization
-  Broken authentication
- Broken object property level authorization
- Unrestricted resource consumption
- Broken function level authorization
- Unrestricted Access to Sensitive Business Flows
- Server Side Request Forgery
- Security Misconfiguration
- Improper Inventory Management
- Unsafe Consumption of APIs

Broken authentication

Brute-Forcing Credentials



Attacker

/authenticate

Authorization: Basic MT0xMjM0NTY=

Authorization: Bearer 324dsf24ergdft435332

Authenticated/Not authenticated



Insecure Shop
REST API Server

Broken authentication

Brute-Forcing Credentials



Attacker

/authenticate



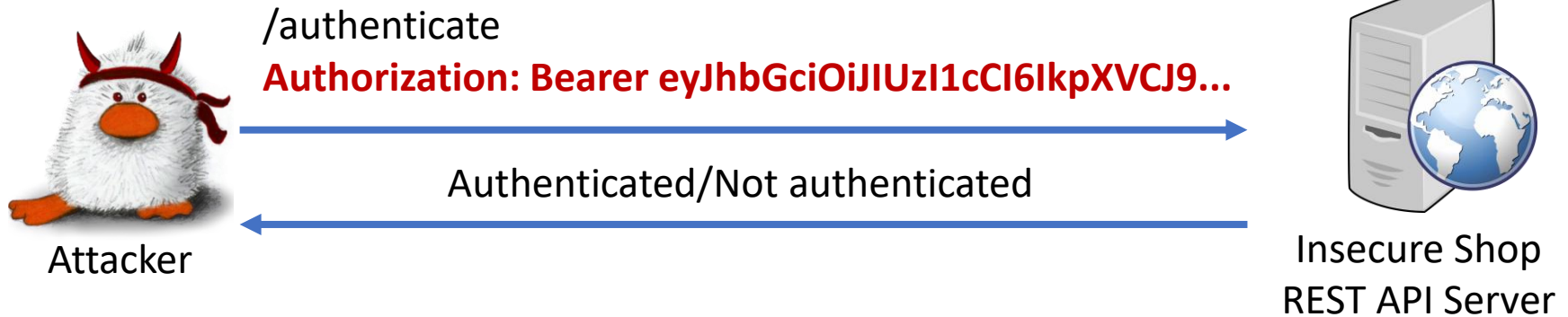
Authenticated



Insecure Shop
REST API Server

Broken authentication

JWT Signature Verification

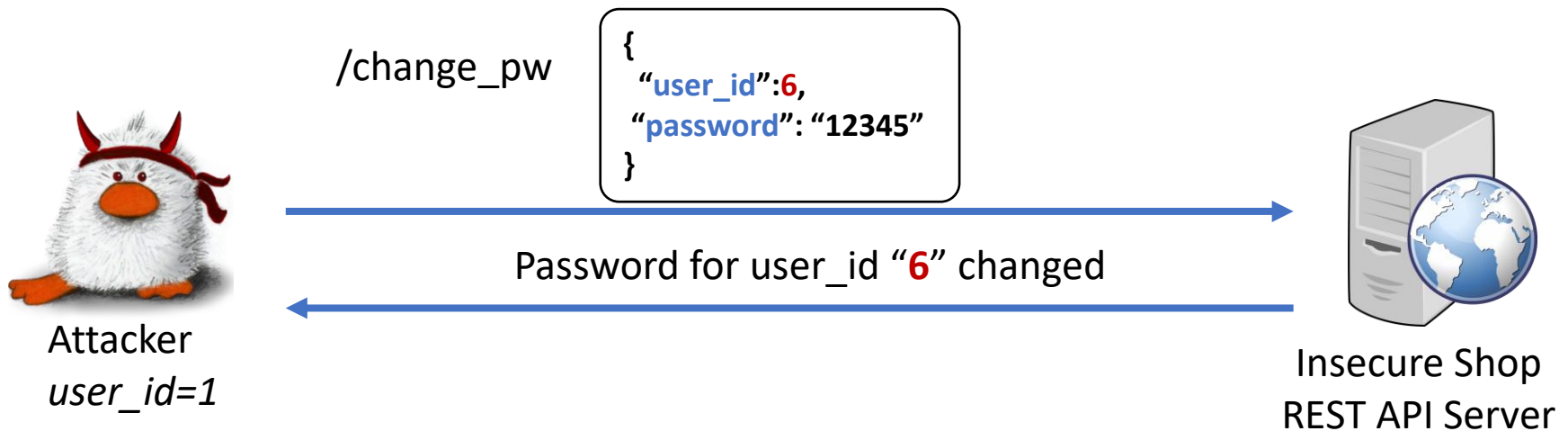


Possible Attacks

1. Insecure Algorithms
2. Key Confusion
3. Custom Keys
4. Server-Side Request Forgery/ Local File Inclusion

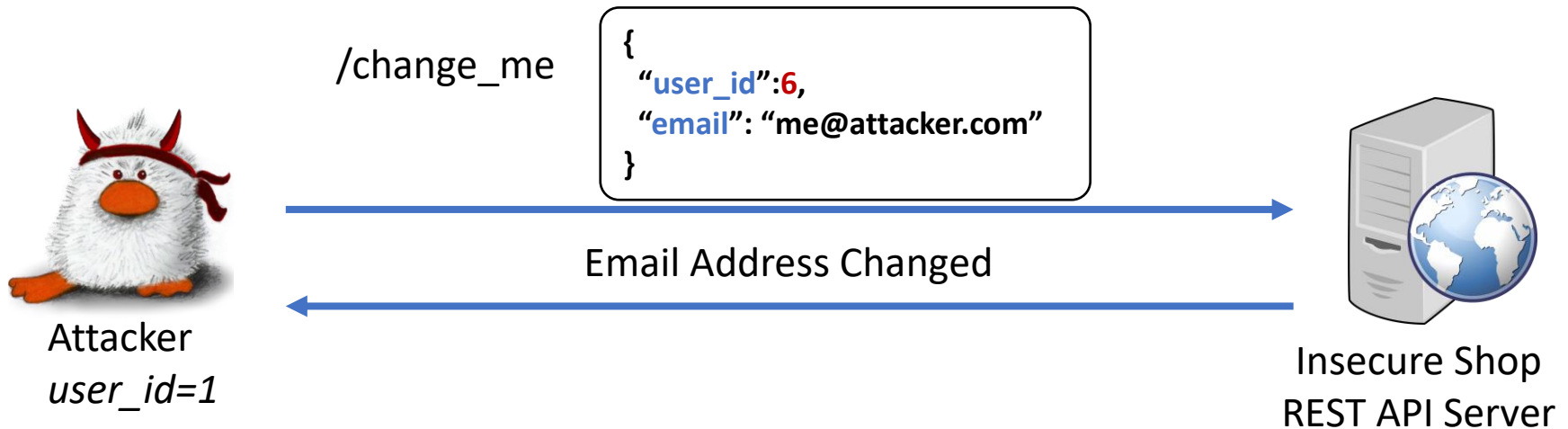
Broken authentication

Changing Login Information



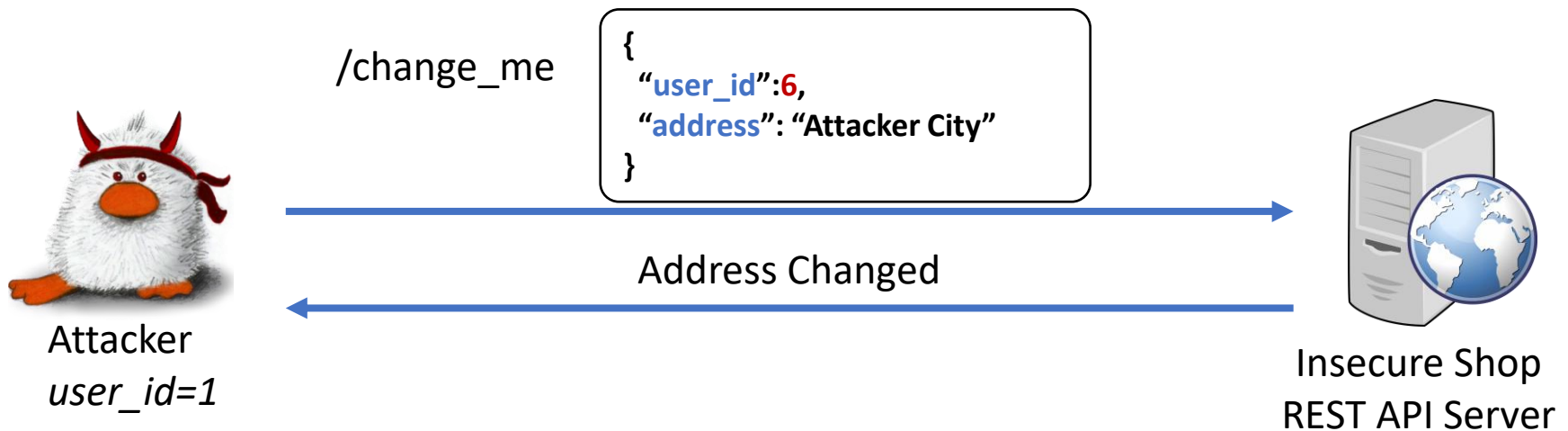
Broken authentication

Changing Login Information



Broken authentication

Changing Login Information



OWASP API Security Top 10

- Broken object level authorization
- Broken authentication
-  Broken object property level authorization
- Unrestricted resource consumption
- Broken function level authorization
- Unrestricted Access to Sensitive Business Flows
- Server Side Request Forgery
- Security Misconfiguration
- Improper Inventory Management
- Unsafe Consumption of APIs

Excessive Data exposure

CLIENT-SIDE FILTERING

The API returns too much data.
This data is filtered on the client side.



Attacker

GET /{shop_id}/{product_id}

All products of all shops

<https://shop.org>

Shows only the results
the user is allowed to
see



shop_id: 1

- product_id: 1
- product_id: n

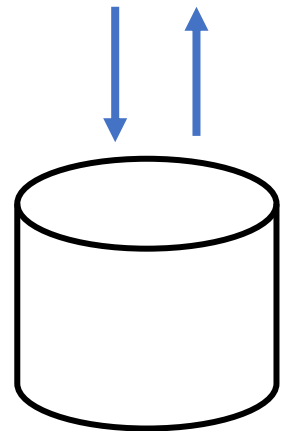
shop_id: 2

- **product_id: x**
- **product_id: y**

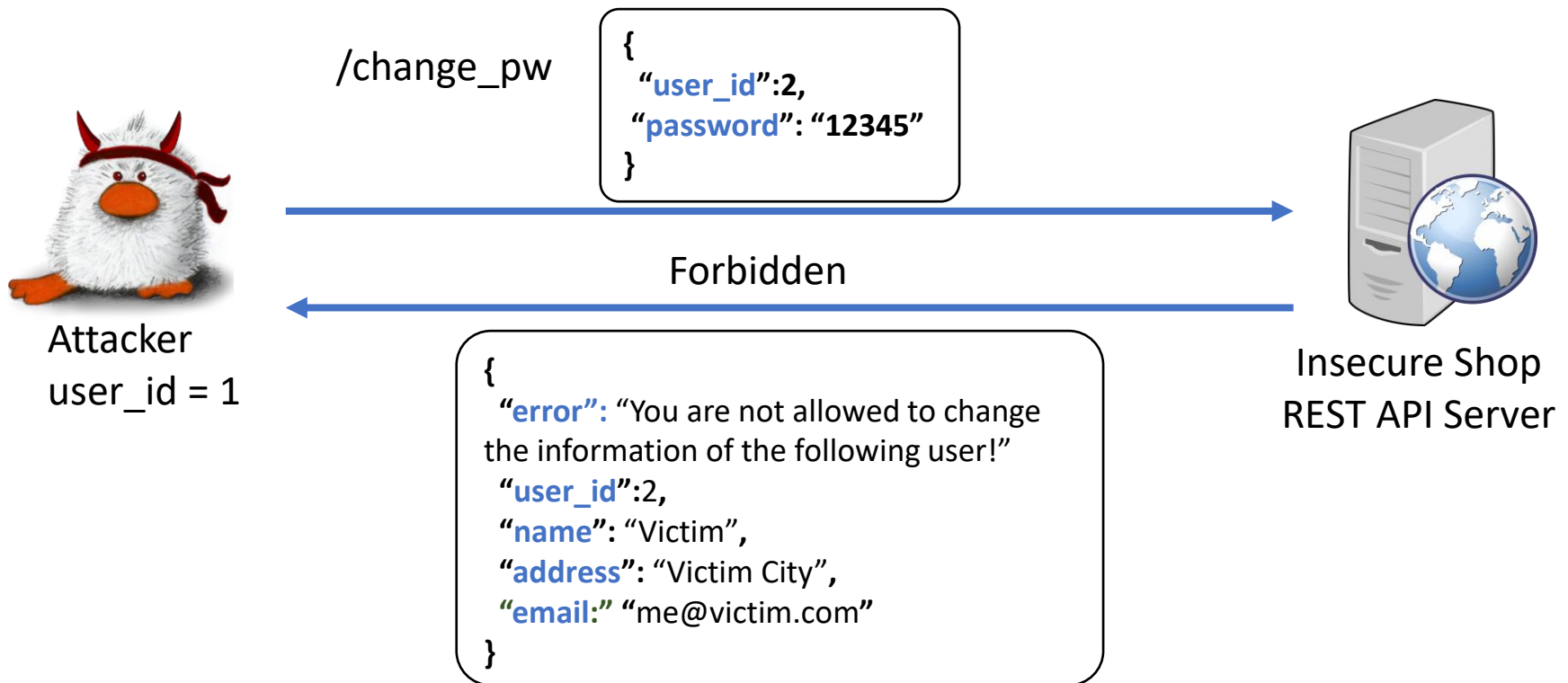
....



Insecure Shop
REST API Server



Excessive Data exposure



Mass assignment

UNPRIVILEGED DATA ASSIGNMENT

Some data should be updated directly by the client and some not.



Customer
user_id = 1

GET /users/{user_id}

```
{ "user_id": 1,  
  "user_pw": "123456" }
```



Insecure Shop
REST API Server

```
POST /users/{user_id}  
{ "user_id": 1,  
  "user_pw": "123456",  
  "user_role": "admin" }
```

Welcome to the **admin** team **user_id = 1**

OWASP API Security Top 10

- Broken object level authorization
- Broken authentication
- Broken object property level authorization
-  • Unrestricted resource consumption
- Broken function level authorization
- Unrestricted Access to Sensitive Business Flows
- Server Side Request Forgery
- Security Misconfiguration
- Improper Inventory Management
- Unsafe Consumption of APIs

Unrestricted resource consumption

API requests consume resources such as:
network, CPU, memory, and storage.



Attacker



Insecure Shop
REST API Server

Service not available



Unrestricted resource consumption

Processing users' files could be dangerous



Seller
user_id =1

PUT /{shop_id}/{product_id}

images=PNG/**JPG**/SVG/XML/PS/PDF/ZIP/* files



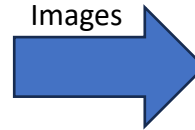
Insecure Shop
REST API Server

Upload Lange Image



Microsoft Image Creator, DALL-E 3

Create
Thumbnail
Images



**CPU/Memory
allocation**

Unrestricted resource consumption

Uploading arbitrary files could be dangerous



Seller
user_id = 1

PUT /{shop_id}/{product_id}

images=PNG/JPG/SVG/XML/PS/PDF/**ZIP**/* files



Insecure Shop
REST API Server

Upload Large
Files

ISO: 10 GB

Mem: 10 GB

Ratio:
1:1

ZIP Bomb

ZIP: 578 B

Mem: 10 GB

Ratio:
1:18,576,848

Memory allocation

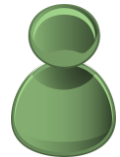
OWASP API Security Top 10

- Broken object level authorization
- Broken authentication
- Broken object property level authorization
- Unrestricted resource consumption
- ➔ Broken function level authorization
- Unrestricted Access to Sensitive Business Flows
- Server Side Request Forgery
- Security Misconfiguration
- Improper Inventory Management
- Unsafe Consumption of APIs

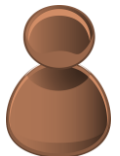
Broken function level authorization

The attackers send **legitimate** API calls to the API endpoint that they **should not have access** to.

Broken function level authorization



Seller



Admin

Servers

Authorize 

Customer Area 

GET 

POST  

Seller Area 

GET  

PUT  

Admin Area 

GET  

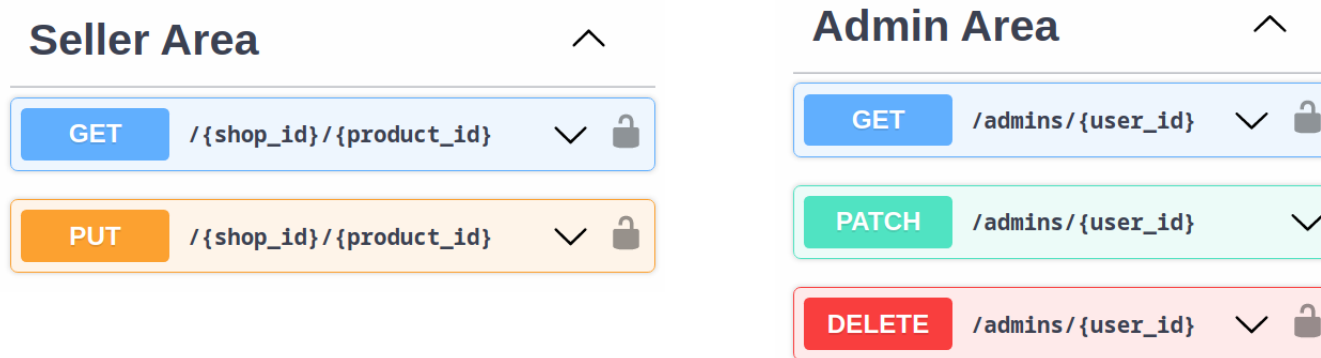
PATCH 

DELETE  



Insecure Shop
REST API Server

Broken function level authorization



Systematic Analysis

- Do operations enforce properly the authentication?
- Are alternative HTTP methods supported?
 - Do hidden paths exist?



Attacker

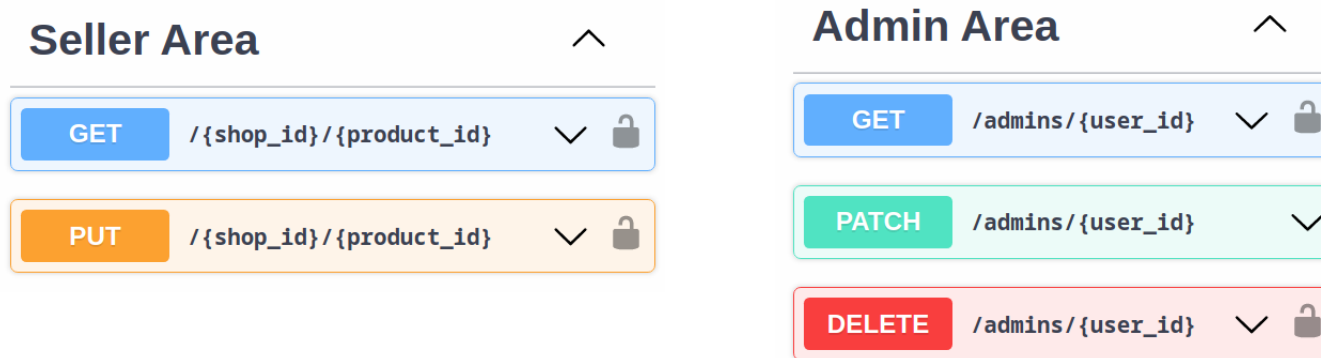
GET `/shop_id/{product_id}`
Authorization: Bearer {wrong_value}
Authorization: Bearer ____
Authorization: ____

Authorization: 0x00
...



Insecure Shop
REST API Server

Broken function level authorization



Systematic Analysis

- Do operations enforce properly the authentication?
- ➔ Are alternative HTTP methods supported?
- Do hidden paths exist?



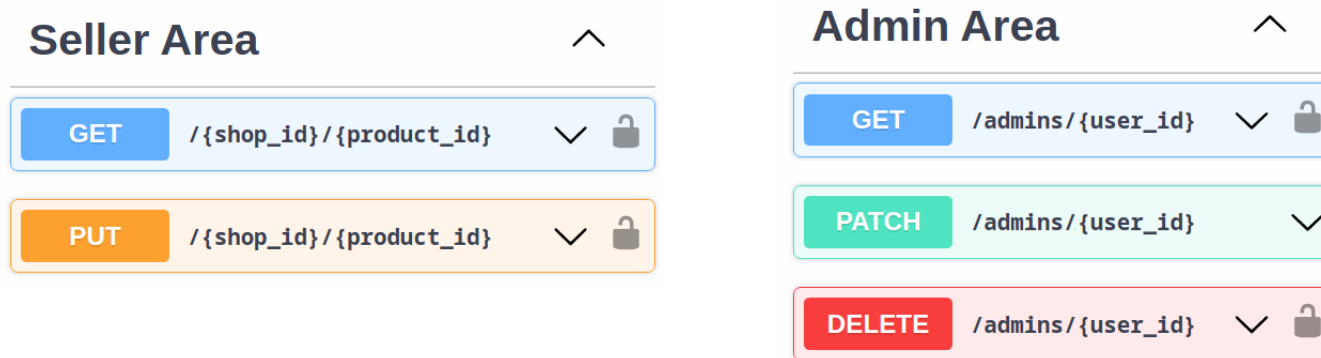
Attacker

DELETE `/shop_id/{product_id}`
PATCH `/shop_id/{product_id}`
... `/shop_id/{product_id}`



Insecure Shop
REST API Server

Broken function level authorization



Systematic Analysis

- Do operations enforce properly the authentication?
- Are alternative HTTP methods supported?

➡ Do hidden paths exist?



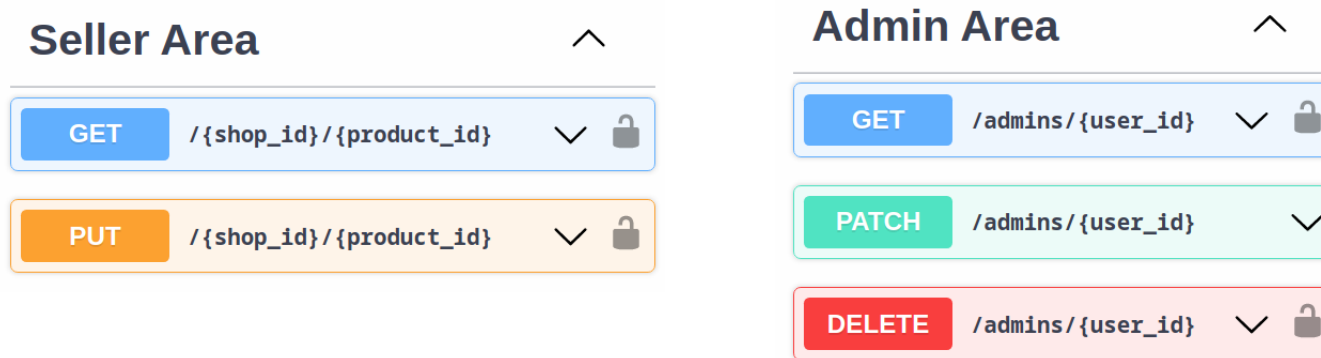
Attacker

```
GET /{shop_id}/*  
GET /{shop_id}/____  
PATCH /users/{user_id}  
PATCH /____/{user_id}
```



Insecure Shop
REST API Server

Broken function level authorization



Systematic Analysis

- Do operations enforce properly the authentication?
- Are alternative HTTP methods supported?

➡ Do hidden paths exist?



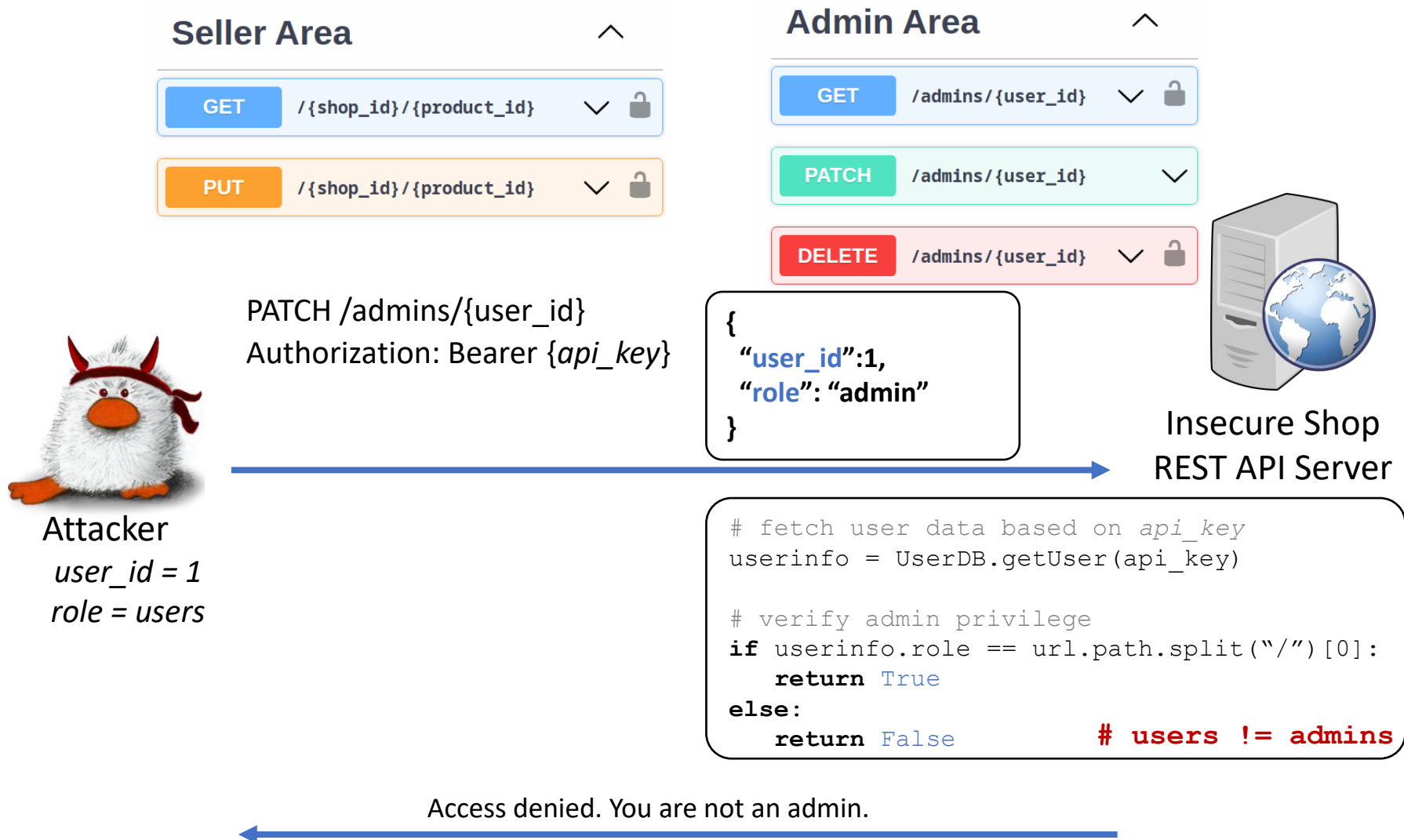
Attacker

GET `/shop_id/`
GET `/shop_id/`
➡ PATCH `/users/{user_id}`
PATCH `/``{user_id}`

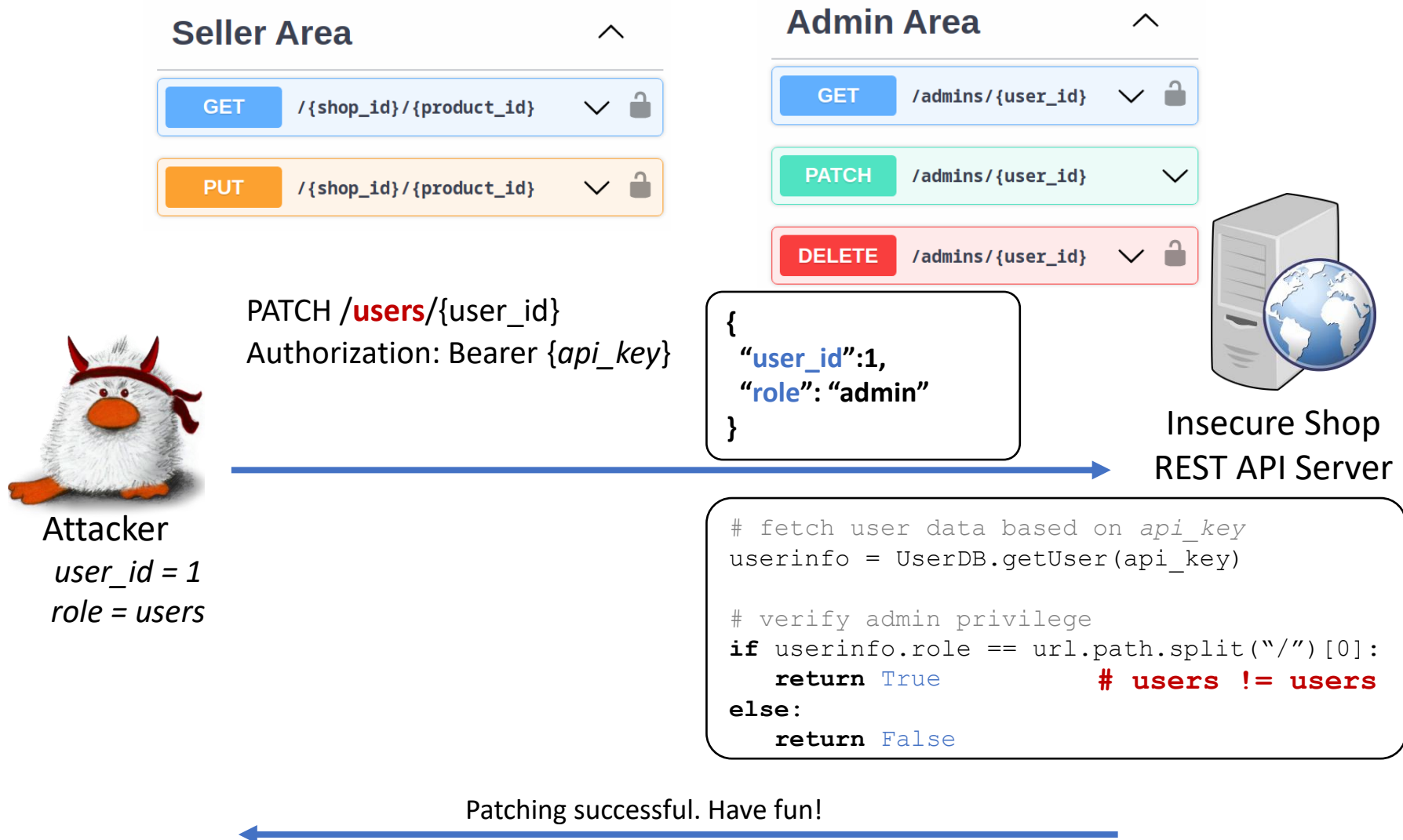


Insecure Shop
REST API Server

Broken function level authorization



Broken function level authorization



OWASP API Security Top 10

- Broken object level authorization
- Broken authentication
- Broken object property level authorization
- Unrestricted resource consumption
- Broken function level authorization
-  Unrestricted Access to Sensitive Business Flows
- Server Side Request Forgery
- Security Misconfiguration
- Improper Inventory Management
- Unsafe Consumption of APIs

Unrestricted Access to Sensitive Business Flows



NEW iPhone to buy. Costs -- 500€!
We have 1.000.000 phones for you.

POST /buy

Authorization: Basic
Base64(1:123456)

```
{  
  "product_id": iPhone  
  "user_id": 1,  
  "quantity": 1.000.000  
}
```



Customer
user_id = 1

Ordered Successful. Thank you!



Insecure Shop
REST API Server



Customer
user_id = 1

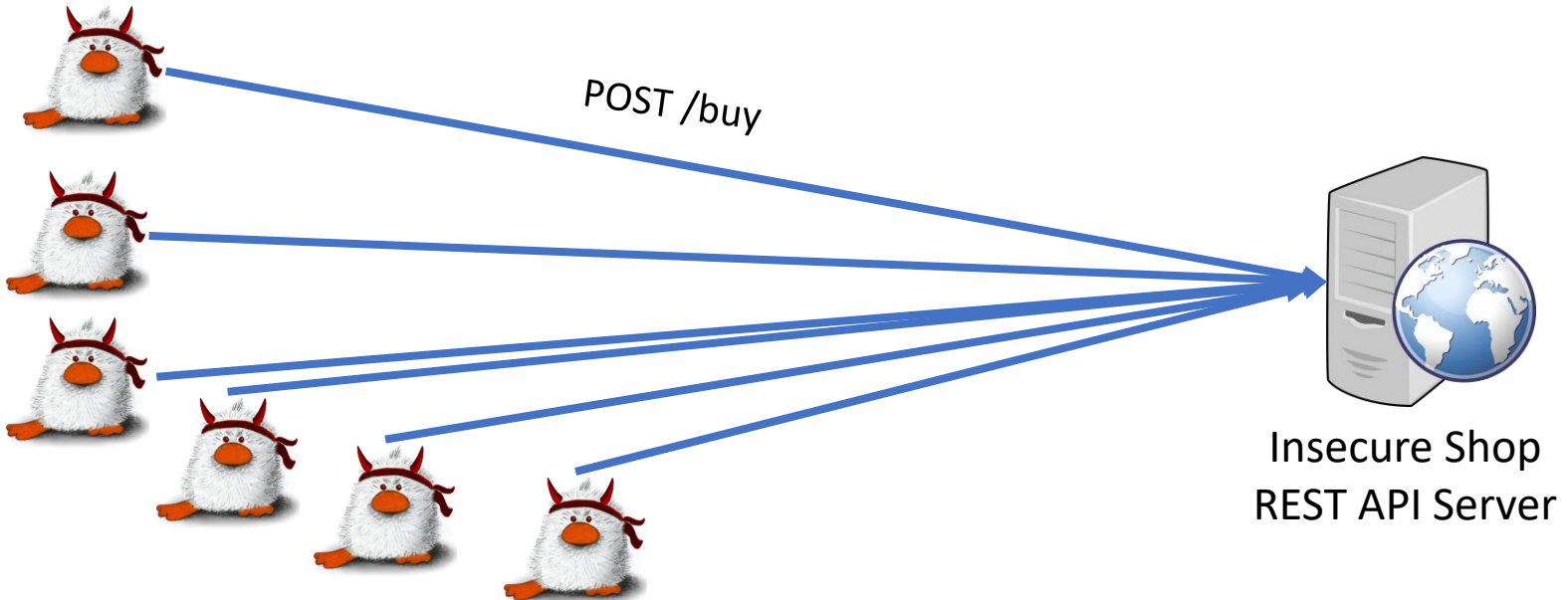


NEW iPhone to buy. Costs – 1.000€!
We have 1.000.000 phones for you.

Unrestricted Access to Sensitive Business Flows



NEW iPhone to buy. Costs -- 500€!
We have 1.000.000 phones for you.

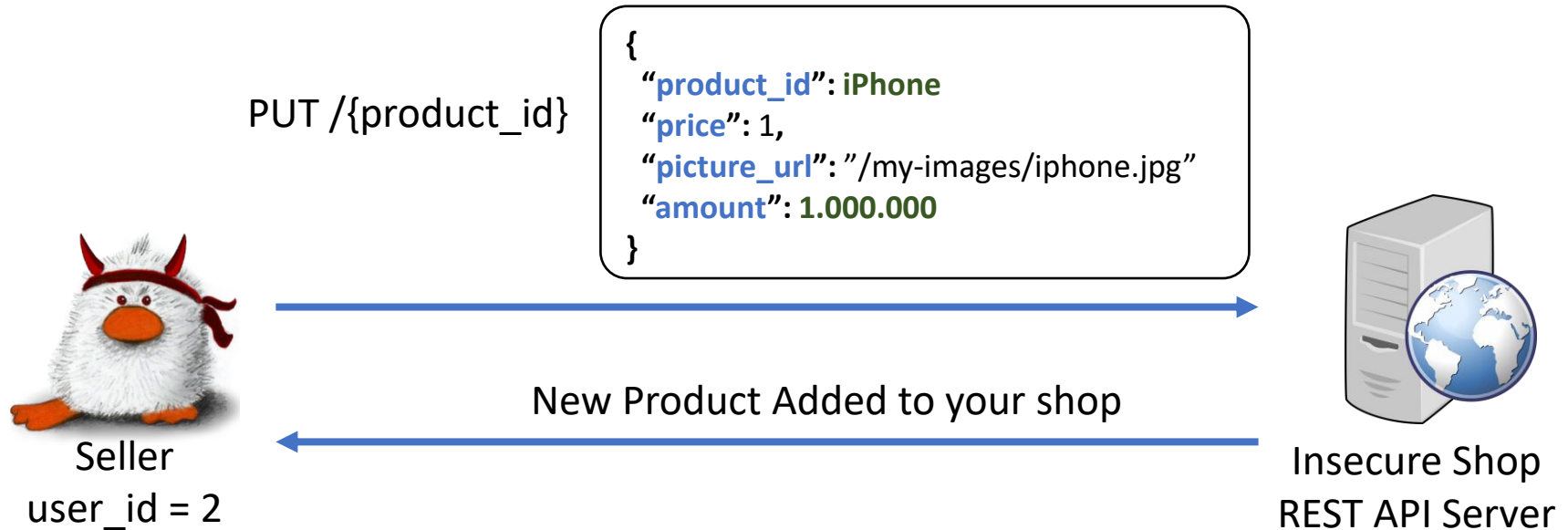


NEW iPhone to buy. Costs – 1.000€!
We have 1.000.000 phones for you.

OWASP API Security Top 10

- Broken object level authorization
- Broken authentication
- Broken object property level authorization
- Unrestricted resource consumption
- Broken function level authorization
- Unrestricted Access to Sensitive Business Flows
- ➡ Server Side Request Forgery
- Security Misconfiguration
- Improper Inventory Management
- Unsafe Consumption of APIs

Server Side Request Forgery



Server Side Request Forgery: DoS



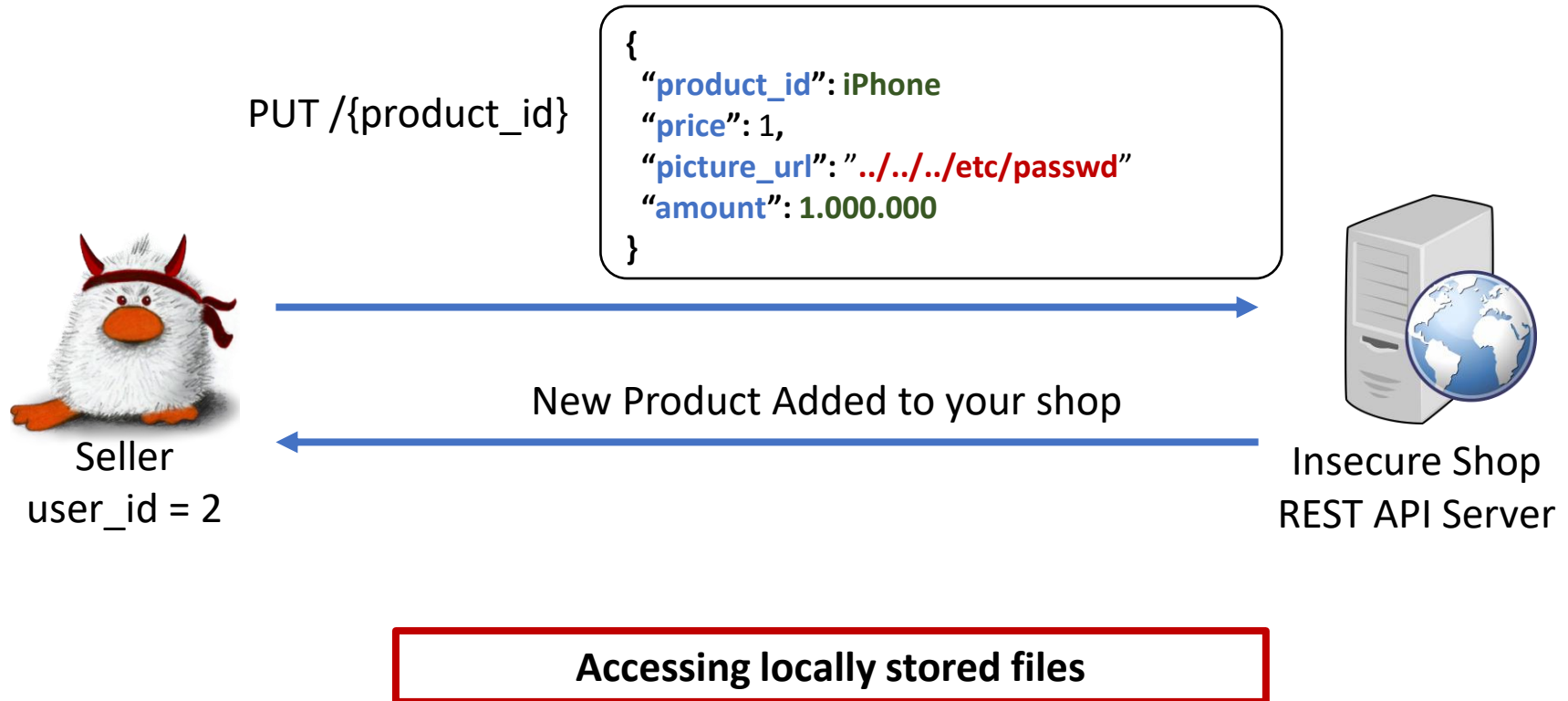
Server Side Request Forgery: PortScanner



Based on the response time, the attacker can figure out whether the port is open or not.

PortScanning is possible

Server Side Request Forgery: Local File Inclusion



Server Side Request Forgery: Local File Inclusion



SSRF ... Real Threat

January 14, 2014 ·

RESEARCH

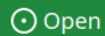
WEBSEC

SSRF

Accessing PayPal's internal network - the critical nature of SSRF

<https://shubs.io/demonstrating-the-critical-nature-of-ssrf---accessing-paypals-internal-network-2/>

Blind SSRF due lack of access controls via `/v1/billing/plans`
#277



Open

imhunterand opened this issue on Jul 24, 2022 · 0 comments

<https://github.com/paypal/gimel/issues/277>

Vimeo upload function SSRF



Sayed Abdelhafiz · Follow

4 min read · Dec 13, 2019

Steal an OAuth Token via SSRF

Extracted from the GitLab blog post "[Tutorial on privilege escalation and post exploitation tactics in Google Cloud Platform environments](#)" by [Chris Moberly](#)

If you've found an SSRF vulnerability on a server hosted in GCP, you can extract this OAuth token from the metadata service if you're lucky, your target has not disabled the `/v1beta` metadata endpoint. Try this first by pointing your SSRF payload at:

```
http://metadata.google.internal/computeMetadata/v1beta/instance/service-accounts/default/token
```


OWASP API Security Top 10

- Broken object level authorization
- Broken authentication
- Broken object property level authorization
- Unrestricted resource consumption
- Broken function level authorization
- Unrestricted Access to Sensitive Business Flows
- Server Side Request Forgery
-  Security Misconfiguration
- Improper Inventory Management
- Unsafe Consumption of APIs

Security Misconfiguration

- Unpatched systems
- Unprotected files and directories
- Unhardened images
- Missing, outdated, or misconfigured TLS
- Error messages with stack traces
- Unnecessary features enabled
- Exposed storage or server management panels
- Missing CORS policy or security headers

OWASP API Security Top 10

- Broken object level authorization
- Broken authentication
- Broken object property level authorization
- Unrestricted resource consumption
- Broken function level authorization
- Unrestricted Access to Sensitive Business Flows
- Server Side Request Forgery
- Security Misconfiguration
- ➔ Improper Inventory Management
- Unsafe Consumption of APIs

Improper Inventory Management

Old API versions are usually **unpatched** and are an **easy way** to compromise systems.

You don't have to fight the most recent API versions.

Servers

<https://shop.mls.org/v2.0/production> ▼

Authorize



Systematic Analysis

➡ API Version

- Alternative API Branches
- Hidden Parameters



Attacker

<https://shop.mls.org/v1.0/production/...>
<https://shop.mls.org/v1.1/production/...>
<https://shop.mls.org/...../production/...>



Insecure Shop
REST API Server

Real Threat?

Steal an OAuth Token via SSRF

Extracted from the GitLab blog post "[Tutorial on privilege escalation and post exploitation tactics in Google Cloud Platform environments](#)" by [Chris Moberly](#)

If you've found an SSRF vulnerability on a server hosted in GCP, you can extract this OAuth token from the metadata server. If you're lucky, your target has not disabled the `/v1beta` metadata endpoint. Try this first by pointing your SSRF payload at:

```
http://metadata.google.internal/computeMetadata/v1beta/instance/service-accounts/default/token
```

However, most folks will have now disabled that old endpoint in favor of the `more secure v1 endpoint`, which requires a custom header to be set. If this is the case, you'll be redirected to `/v1/` and given an HTTP 403 error. You'll need to find a way to set the header as described in the [metadata page](#).

<https://hackingthe.cloud/gcp/exploitation/gcp-metadata-ssrf/>

Improper Inventory Management

Old API versions are usually **unpatched** and are an **easy way** to compromise systems.

You don't have to fight the most recent API versions.

Servers

<https://shop.mls.org/v2.0/production> ▼

Authorize



Systematic Analysis

- API Version
- ➔ Alternative API Branches
- Hidden Parameters



Attacker

[https://shop.mls.org/v2.0/development/...](https://shop.mls.org/v2.0/development/)
[https://shop.mls.org/v2.0/test/...](https://shop.mls.org/v2.0/test/)
[https://test.shop.mls.org/v2.0/...](https://test.shop.mls.org/v2.0/)

Hard to guess



Insecure Shop
REST API Server

Improper Inventory Management

Old API versions are usually **unpatched** and are an **easy way** to compromise systems.

You don't have to fight the most recent API versions.

Servers

<https://shop.mls.org/v2.0/production> ▼

Authorize



Systematic Analysis

- API Version
- Alternative API Branches

➡ Hidden Parameters



Attacker

<https://shop.mls.org/v2.0/production/...?debug=true>
<https://shop.mls.org/v2.0/production/...?development=1>
<https://shop.mls.org/v2.0/production/...?...>

Hard to guess



Insecure Shop
REST API Server

OWASP API Security Top 10

- Broken object level authorization
 - Broken authentication
 - Broken object property level authorization
 - Unrestricted resource consumption
 - Broken function level authorization
 - Unrestricted Access to Sensitive Business Flows
 - Server Side Request Forgery
 - Security Misconfiguration
 - Improper Inventory Management
-  Unsafe Consumption of APIs

Lessons Learned Until Now?

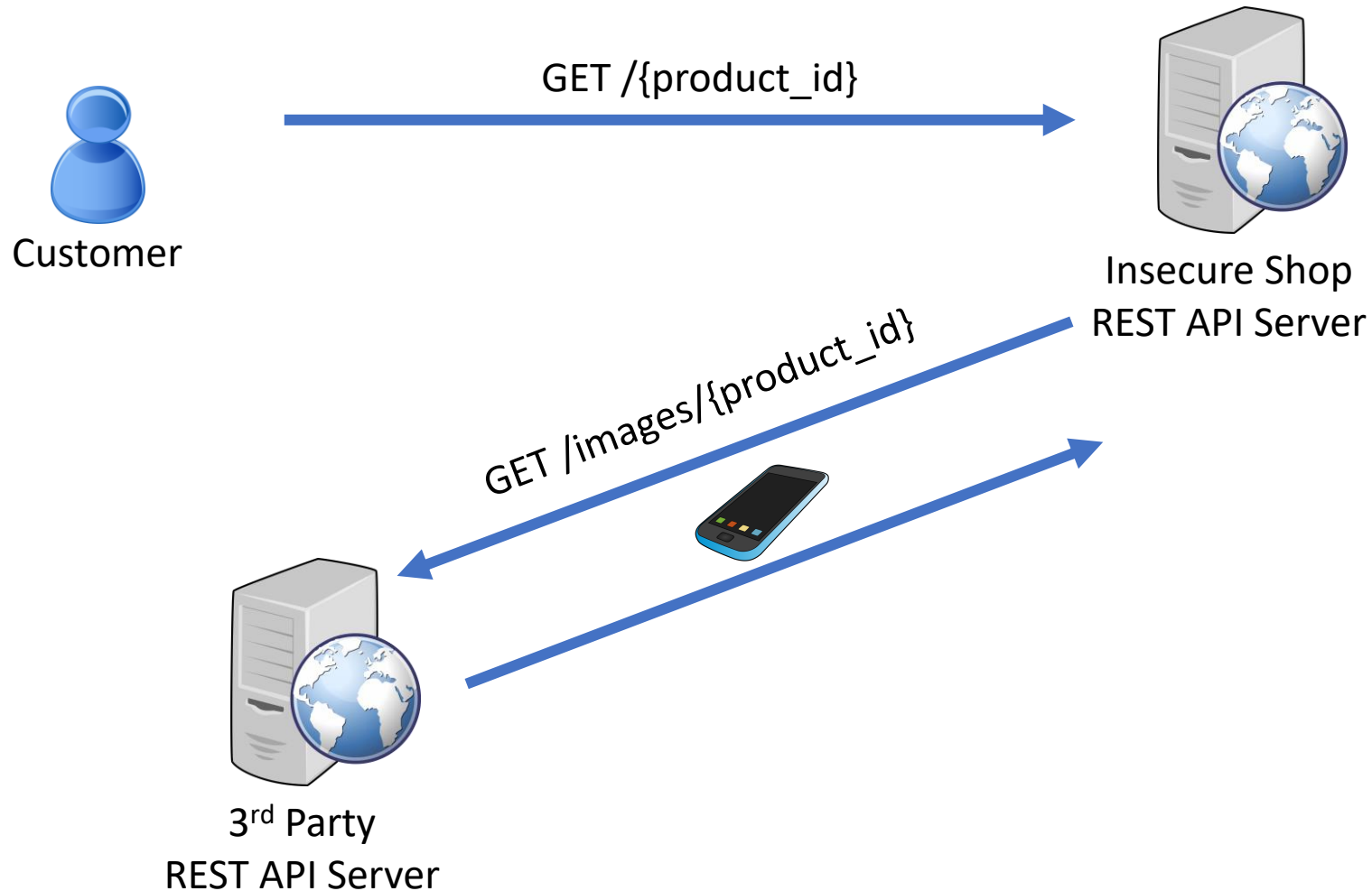


Do not trust user input!!!

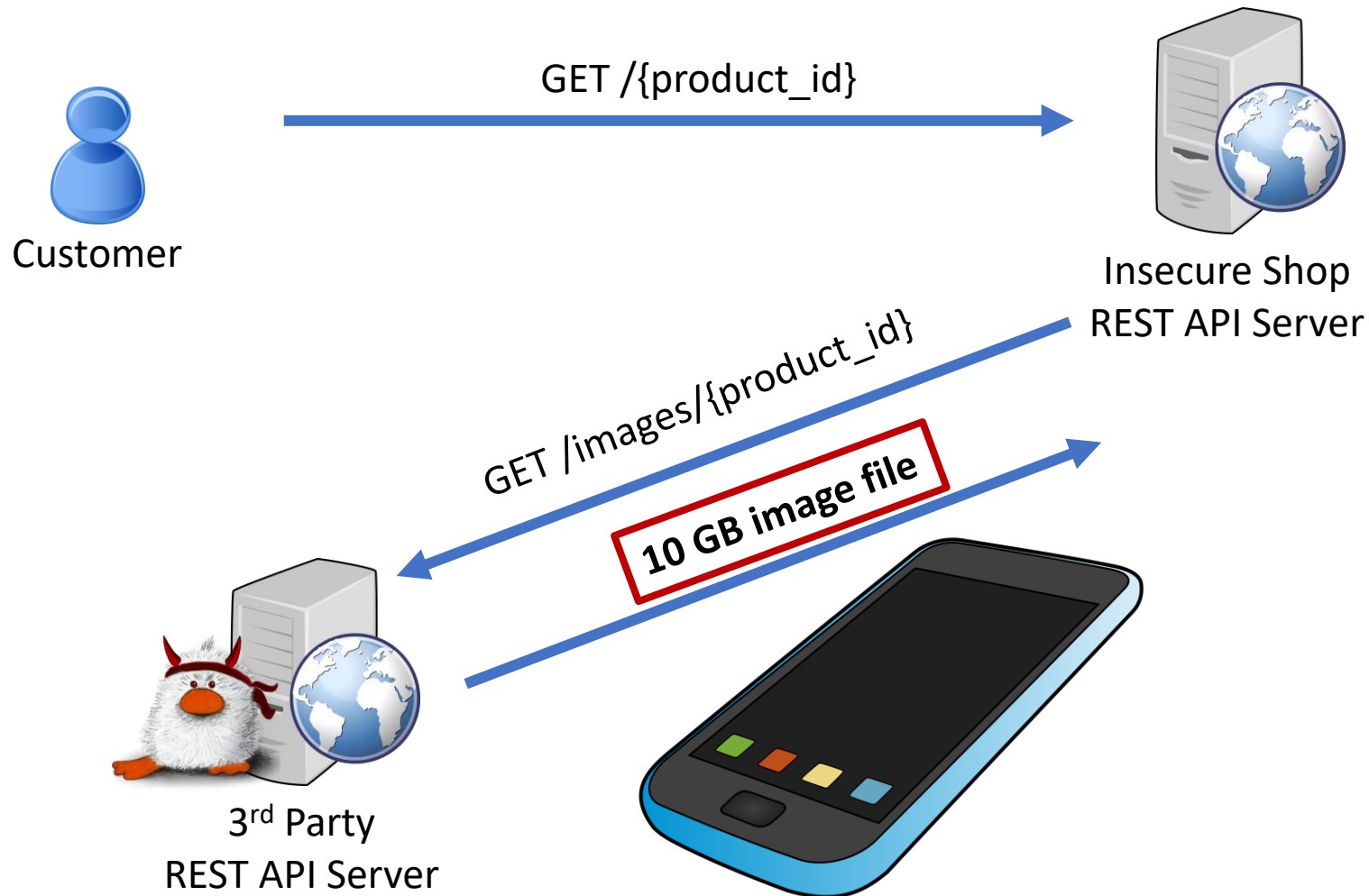


But other APIs are OK?

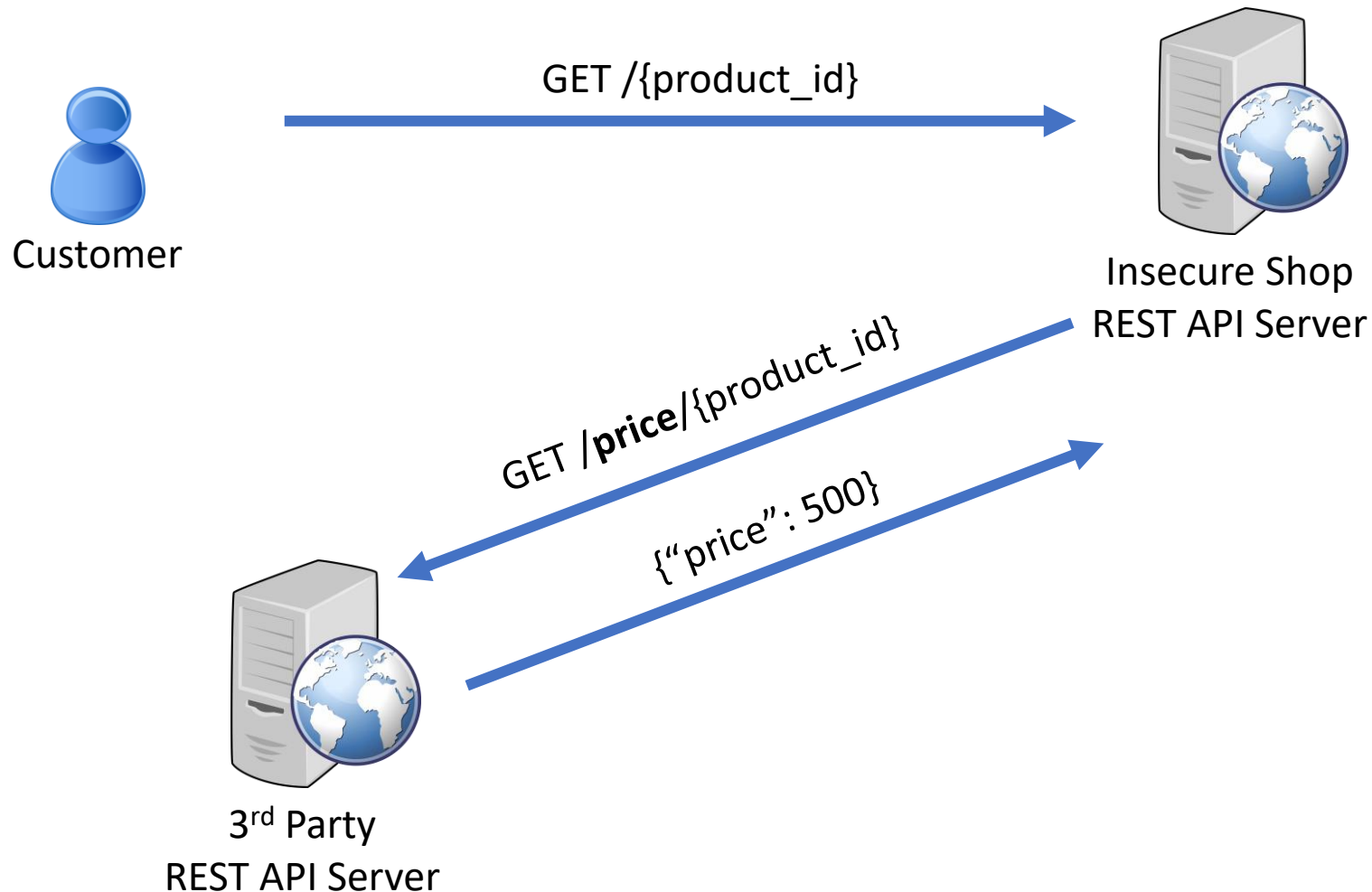
Unsafe Consumption APIs



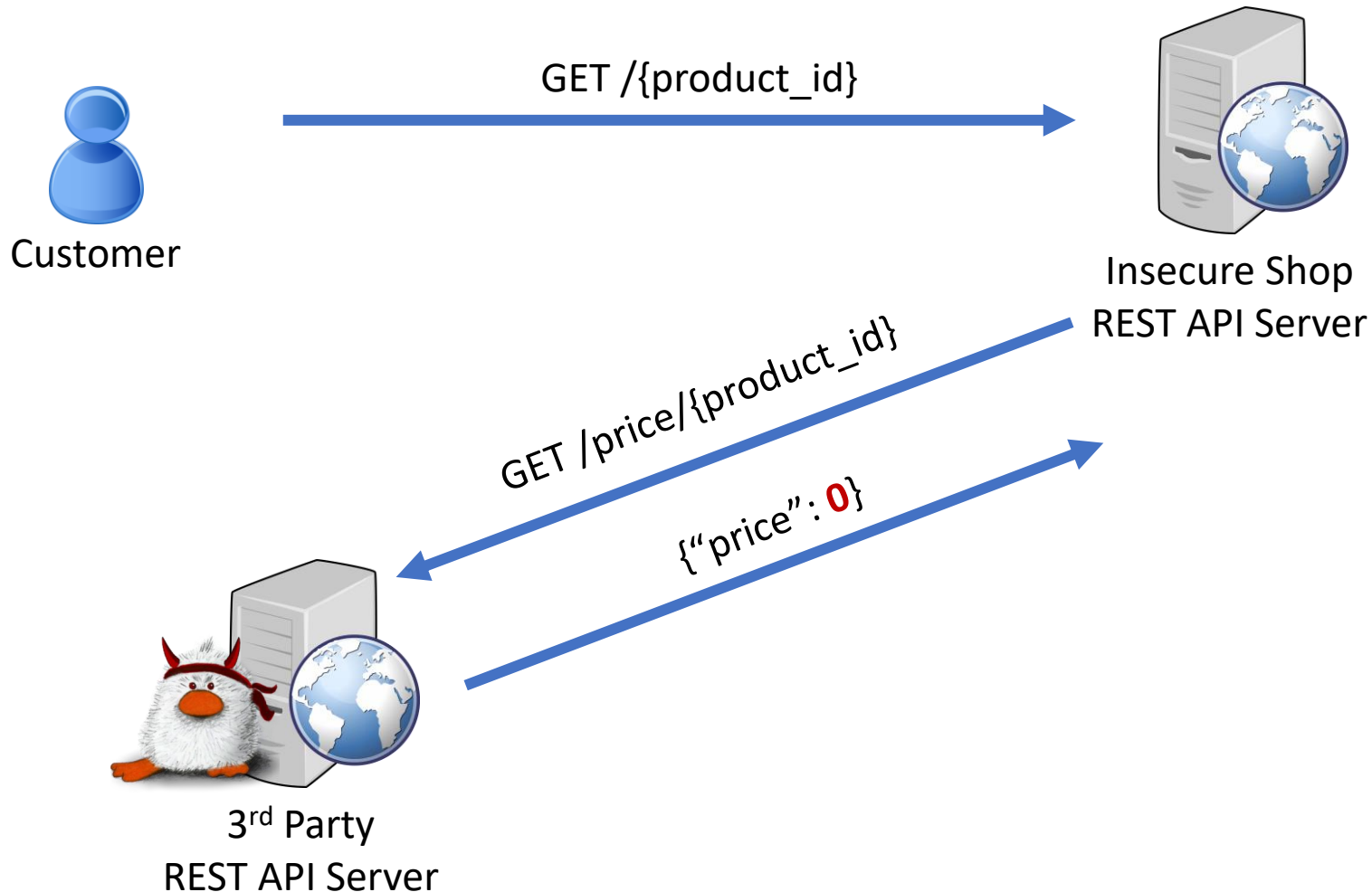
Unsafe Consumption APIs: Denial-of-Service



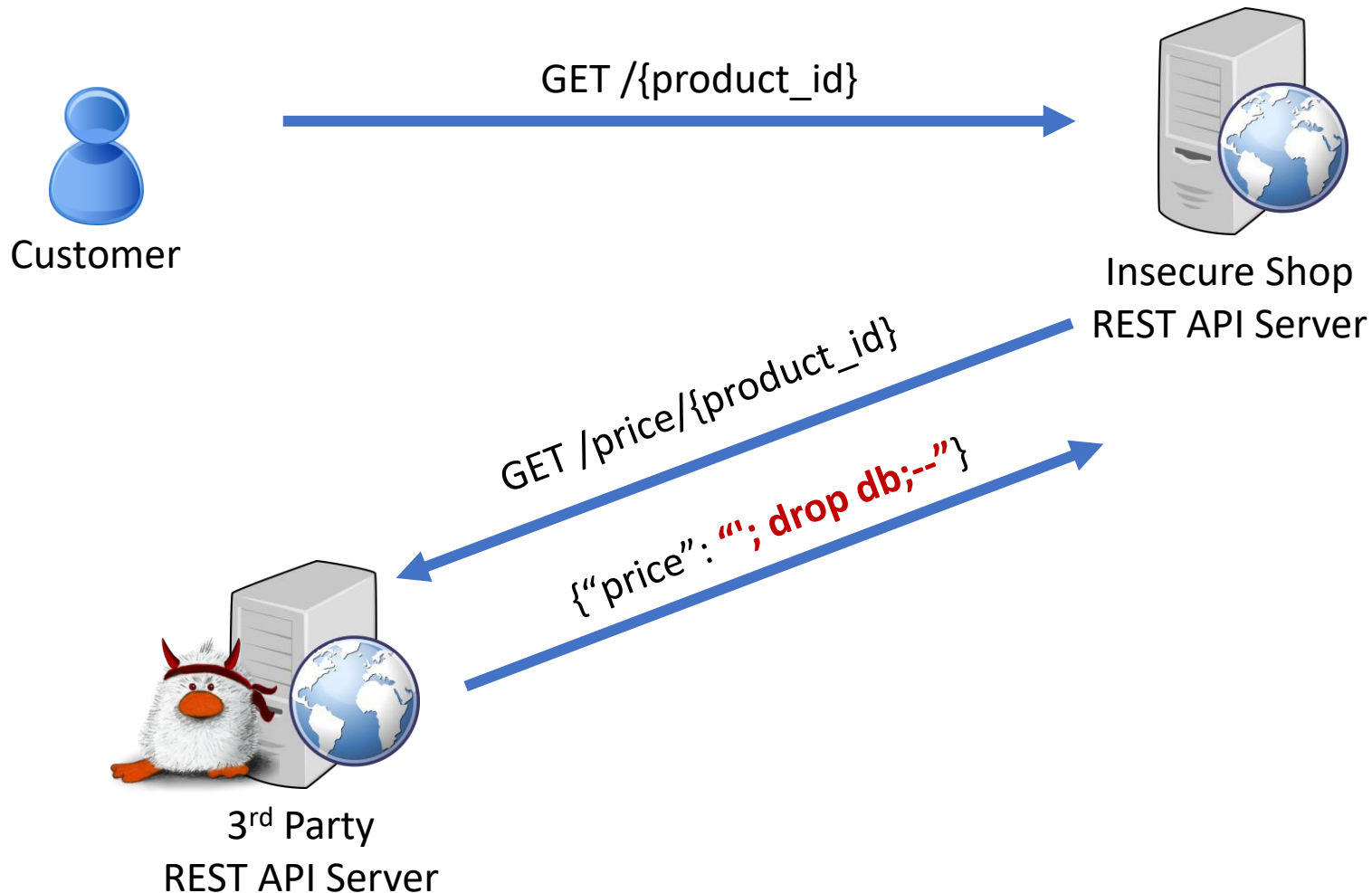
Unsafe Consumption APIs



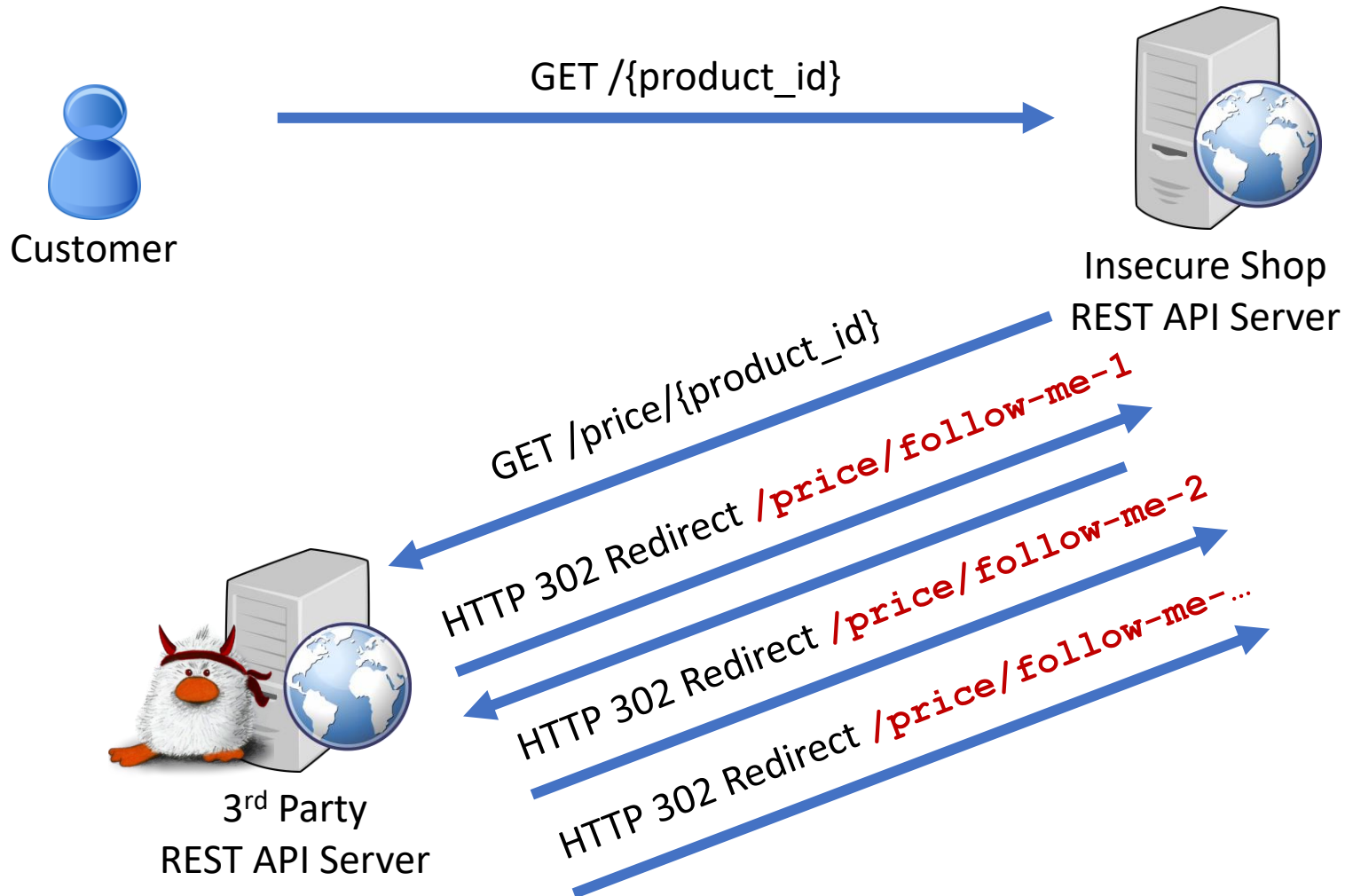
Unsafe Consumption APIs: Sensitive Business Flow



Unsafe Consumption APIs: SQL Injection



Unsafe Consumption APIs: Denial-of-Service



REST API Security

Best Current Practices

Best Practices to Secure REST APIs

Keep it simple

- Always use HTTPs
- Use password hash
- Never expose information on URLs
- Consider Oauth
- Input Parameter Validation

Keep it simple

- Avoid *unclear* and *complex* design
- Avoid exceptions from the *normal* behavior
 - Custom parameter *debug=1*
- Avoid different views on the same data
 - Separation between *verification* vs. *application* logic
- Avoid *redundant* and *superfluous* parameters

Verification Logic:
verify(user_id, password)

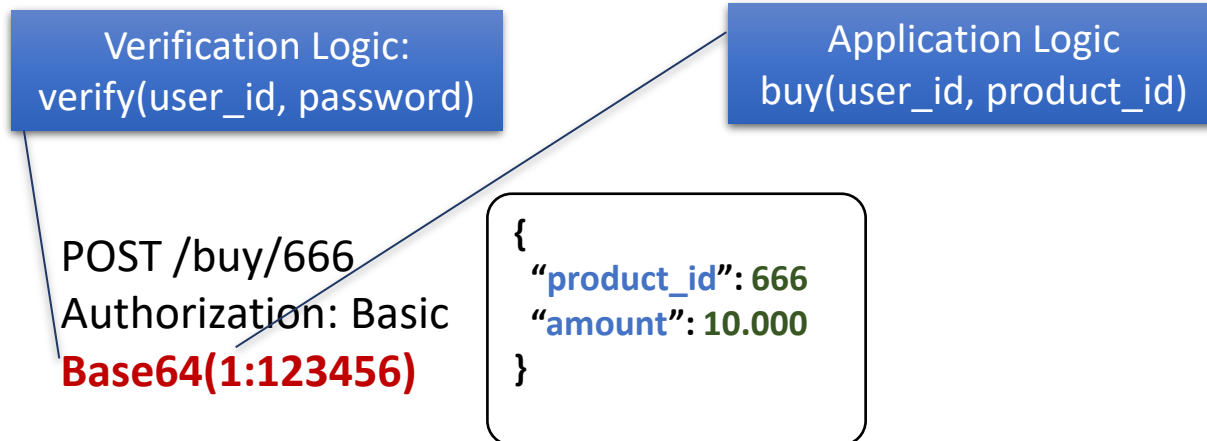
POST /buy/666
Authorization: Basic
Base64(1:123456)

Application Logic
buy(user_id, product_id)

```
{  
  "product_id": 666  
  "user_id": 6,  
  "amount": 10.000  
}
```

Keep it simple

- Avoid *unclear* and *complex* design
- Avoid exceptions from the *normal* behavior
 - Custom parameter *debug=1*
- Avoid different views on the same data
 - Separation between *verification* vs. *application* logic
- Avoid *redundant* and *superfluous* parameters



Best Practices to Secure REST APIs

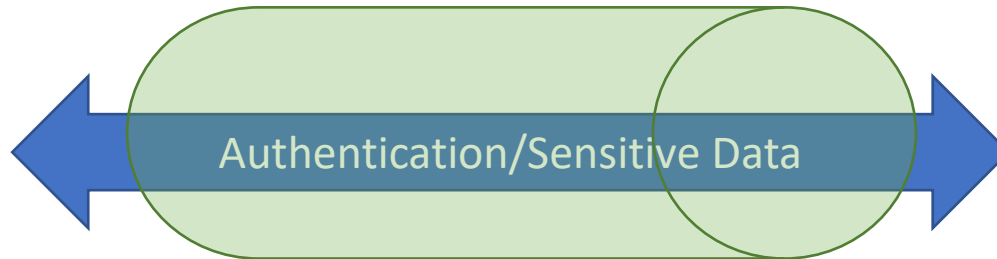
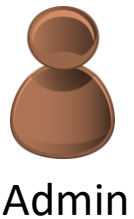
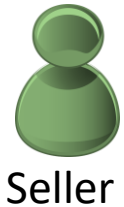
- Keep it simple

 Always use HTTPS

- Use password hash
- Never expose information on URLs
- Consider OAuth
- Input Parameter Validation

BCP: Always use HTTPs

Use TLS



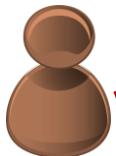
Best Practices to Secure REST APIs

- Keep it simple
- Always use HTTPS
- ➡ Use password hash
- Never expose information on URLs
- Consider OAuth
- Input Parameter Validation

BCP: Use Password Hash



Seller



Admin

Servers

<https://shop.mls.org/v2.0/production> Authorize

Customer Area ^

GET [{product_id}](/{product_id}) ✓

POST </buy> ✓ 

Seller Area ^

GET [{shop_id}/{product_id}](/{shop_id}/{product_id}) ✓ 

PUT [{shop_id}/{product_id}](/{shop_id}/{product_id}) ✓ 

Admin Area ^

GET [{user_id}](/admins/{user_id}) ✓ 

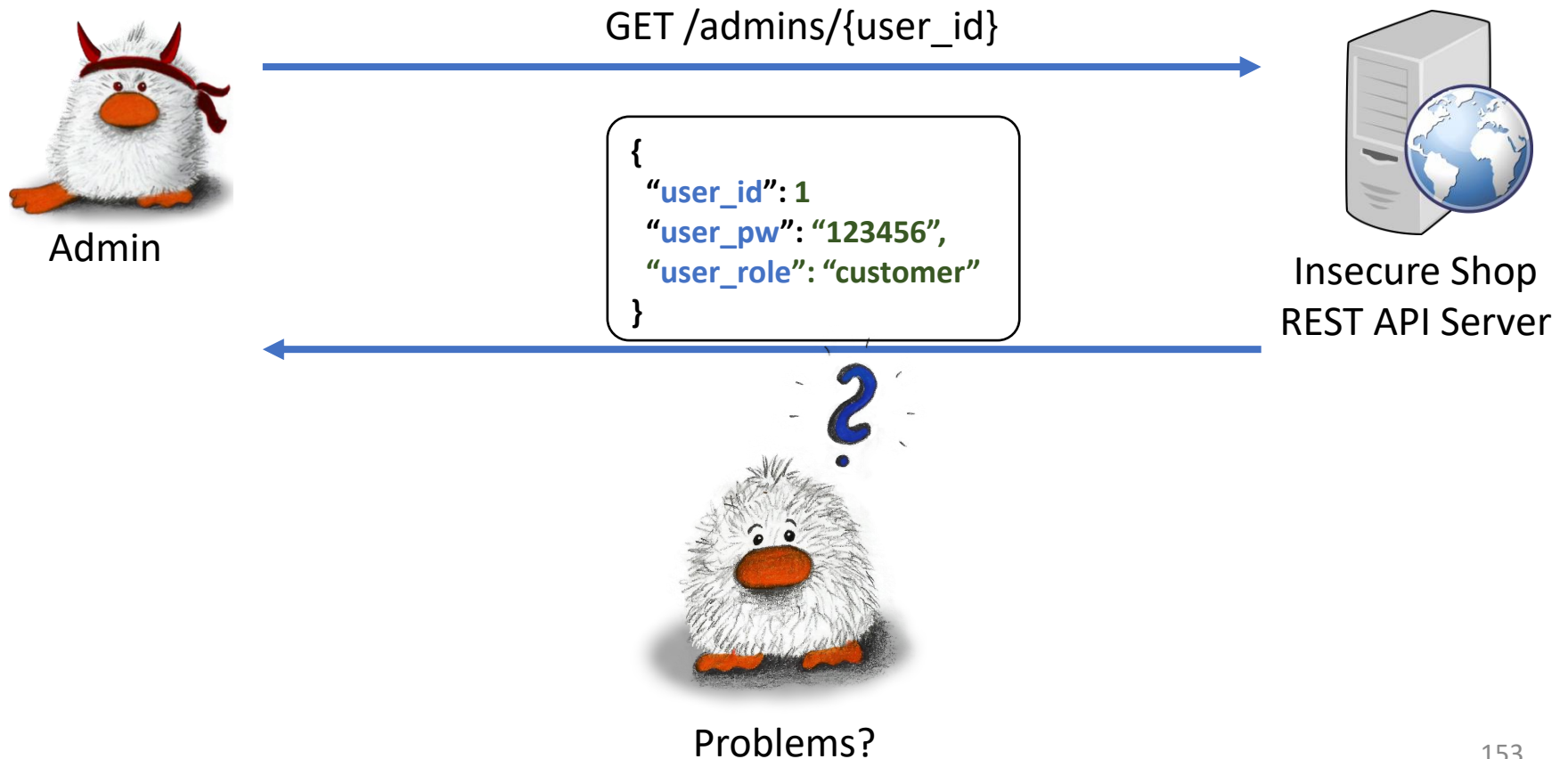
PATCH [{user_id}](/admins/{user_id}) ✓

DELETE [{user_id}](/admins/{user_id}) ✓ 



Insecure Shop
REST API Server

BCP: Use Password Hash



BCP: Use Password Hash

- Protect the API against all potential malicious entities
- Follow the best current practices
 - [Password Storage Cheat Sheet](#)

Best Practices to Secure REST APIs

- Keep it simple
- Always use HTTPS
- Use password hash
- ➡ Never expose information on URLs
- Consider OAuth
- Input Parameter Validation

BCP: Sensitive Information in URL

- Avoid sending sensitive information in the URL
 - For example: tokens, credentials
 - Tokens/API keys in the URL can be logged and misused
 - Bookmarks can leak sensitive data
- Use instead HTTP headers or HTTP Body to transmit sensitive data

Best Practices to Secure REST APIs

- Keep it simple
- Always use HTTPS
- Use password hash
- Never expose information on URLs
- ➔ Consider OAuth
- Input Parameter Validation

More on this topic later

Best Practices to Secure REST APIs

- Keep it simple
- Always use HTTPS
- Use password hash
- Never expose information on URLs
- Consider OAuth

 Input Parameter Validation

Input Parameter Validation

- Never trust your API consumers, even if they are internal.
- Strictly define all input data, such as schemas, types, and string patterns, and enforce them at runtime.
- Validate, filter, and sanitize all incoming data.
- Define, limit, and enforce API outputs to prevent data leaks.

Research Questions

- Where and how to find APIs?
- How can we use them directly and fully automatically?
- Which attacks can be automated?
- How to automate security analysis?
- How to evaluate REST APIs by avoiding harm?

